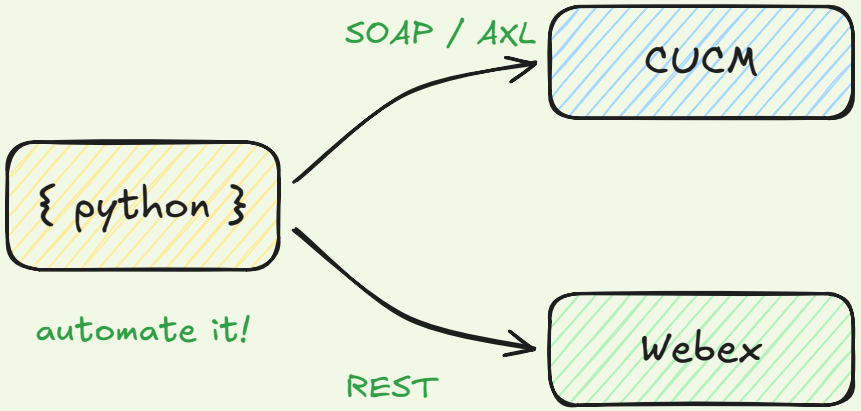
clauto

Cisco Collaboration Automation

Python fundamentals for voice & collaboration engineers



Ilker Mansur - CCIE 63555

2026

Contents

1 Python Fundamentals 3

1.1 Why Python? 4
1.2 Variables and Objects 5
1.3 Data Types 6
1.3.1 int 9
1.3.2 float 10
1.3.3 str 12
1.3.4 bool 16
1.3.5 list 17
1.3.6 tuple 20
1.3.7 range 21
1.3.8 dict 23
1.3.9 set 26
1.3.10 bytes 28
1.3.11 None 30
1.4 if Statements 31
1.5 Loops 34
1.5.1 for 35
1.5.2 while 38
1.6 Functions 40
1.6.1 args and kwargs 43
1.6.2 Modules and import 45
1.7 Exceptions 48
1.8 Files 51
1.8.1 CSV 53
1.8.2 JSON 55
1.9 Comprehensions 56
1.10 Classes 59
1.11 Credentials 62

2 APIs and Libraries 64

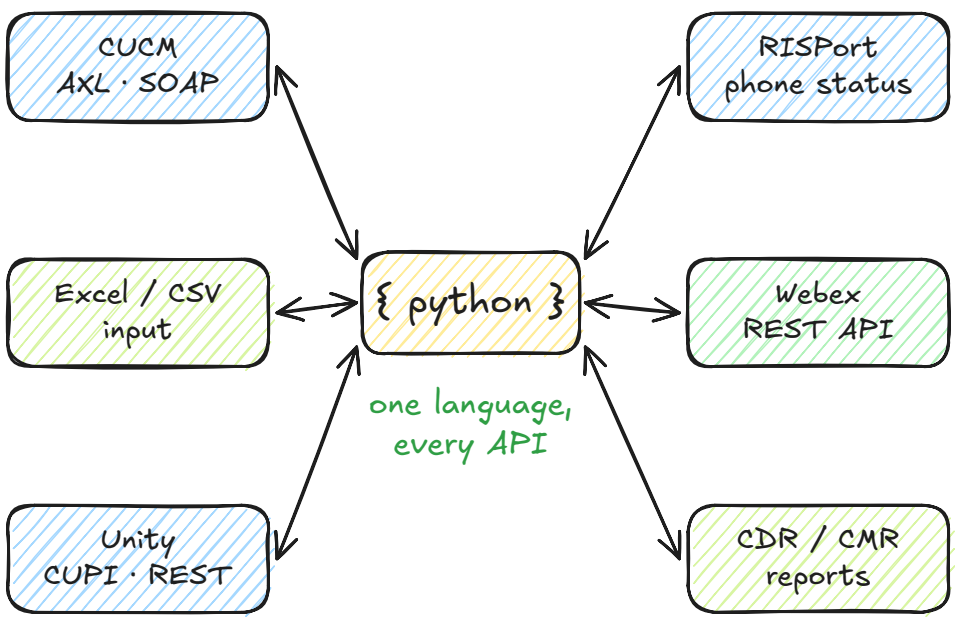
2.1 HTTP Basics 65
2.2 JSON and XML 68
2.3 requests 71
2.4 zeep and AXL 76
2.5 RisPort70 85
2.6 Netmiko 88
2.7 CMS REST API 92
2.8 Remote Phone Control 96
2.9 Wrap-up 99

1 Python Fundamentals

The core language you need before talking to CUCM and Webex: data types, control flow, functions and files. Every example comes from the collaboration world, so the syntax and the use case are learned together.

1.1 Why Python?
1.2 Variables and Objects
1.3 Data Types
1.4 if Statements
1.5 Loops
1.6 Functions
1.7 Exceptions
1.8 Files
1.9 Comprehensions
1.10 Classes
1.11 Credentials

Python for collaboration engineers



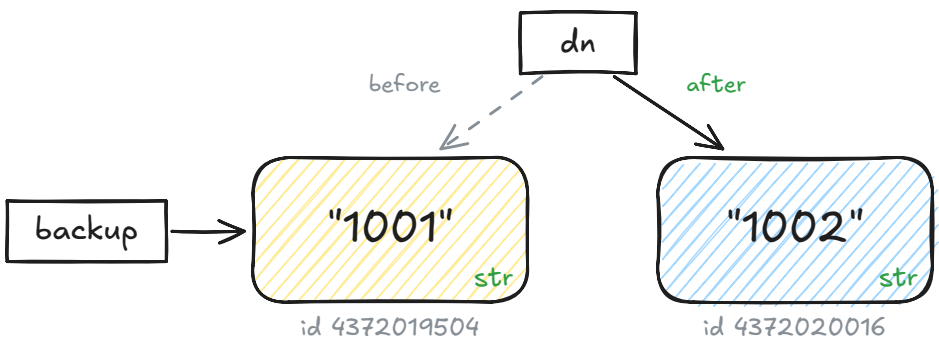
Python is a high-level, general-purpose language with a readable, almost English-like syntax. It is interpreted and dynamically typed: write a few lines, run them, see the result. No compile step, no boilerplate.

Why it matters in Collaboration

- **Scale:** add 500 users and phones with one loop instead of 500 GUI forms.
- **Consistency:** the same partition, CSS and device pool every time, with no typos.
- **Visibility:** registration status, inventory and CDRs in one report, in seconds.
- **Integration:** one language speaks AXL, RIS, CUPI and the Webex REST API.

> ▸ **IN THE COLLAB WORLD**
> Cisco DevNet samples for AXL and Webex are mostly written in Python, so you can reuse them from day one.

Variables are labels, not boxes




```
dn = "1001"
backup = dn             # second label, same box
id(dn), id(backup)      # (4372019504, 4372019504)
dn is backup            # True

dn = "1002"             # dn moves to a new box
backup                  # '1001'
```

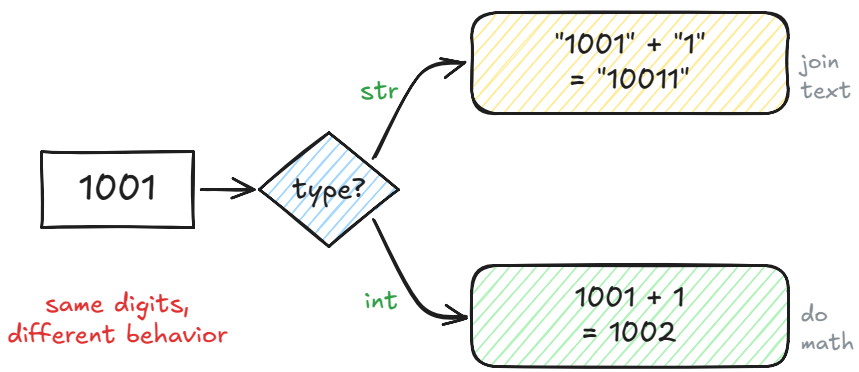
Think of an **object** as a box in memory that holds a value and its type. A **variable** is not the box. It is a **label** tied to a box. Python has no explicit pointers: every name already points to an object.

`=` never copies a value. It ties a label to a box. `backup = dn` gives the same box a second label. `dn = "1002"` unties `dn` and ties it to a new box. The old box is untouched.

`id(x)` returns the box's identity: a unique number for the object's lifetime (its memory address in CPython). `a is b` compares identity, `a == b` compares values.

> ▸ **IN THE COLLAB WORLD**
> `new = phone` on an AXL phone `dict` gives a second label, not a copy. Changing `new["description"]` changes `phone` too. Use `phone.copy()`.

Why do we need data types?




```
"1001" + "1"      # '10011'
1001 + 1          # 1002
"1001" + 1        # TypeError: can only
                  # concatenate str to str
int("0012")       # 12  ← leading zeros lost
```

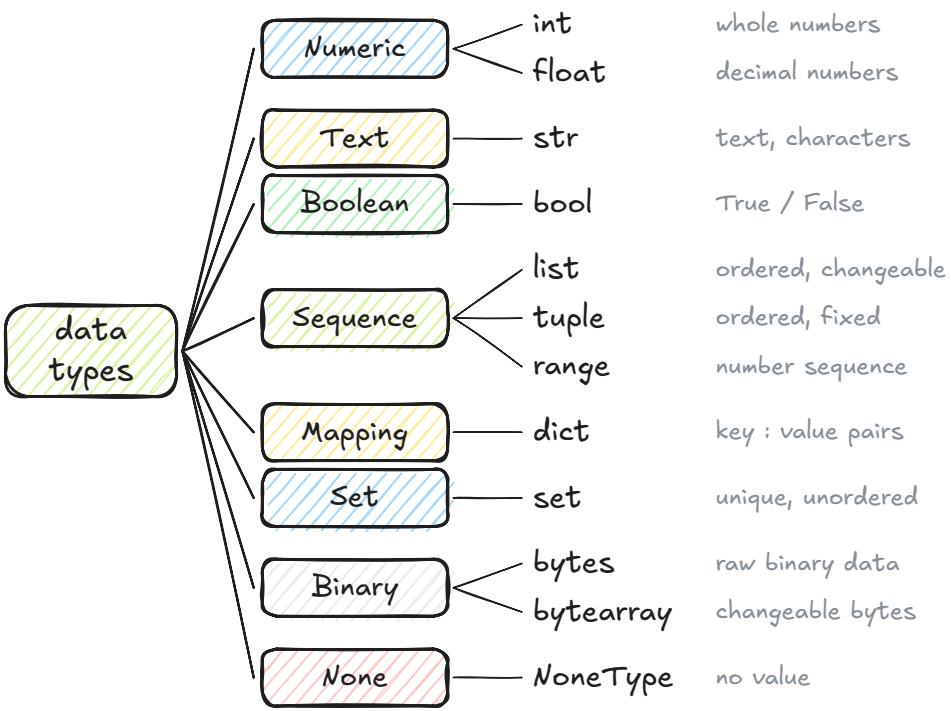
Memory only holds bytes. On its own, `1001` could be a number, a directory number or four characters. A **data type** gives raw data its meaning and answers three questions:

1. **What is it?** Text, a whole number, a yes/no flag, a collection.
2. **What can I do with it?** Add numbers, join text, loop over a list.
3. **How is it shown and stored?** `"0012"` keeps its zeros, `12` does not.

Python is **strongly typed**: it never guesses. `"1001" + 1` raises a `TypeError` instead of silently picking a meaning, so the bug shows up early, not in production.

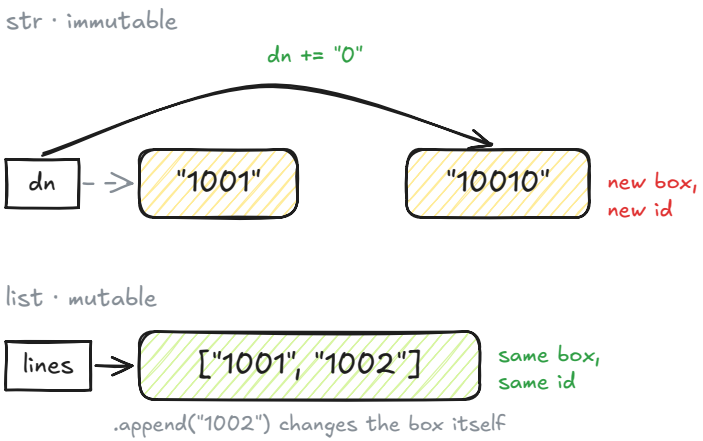
> ▸ **IN THE COLLAB WORLD**
> A DN looks like a number but is a `str`: `0012`, `+905551112233` and `\+90XXX` all break as `int`. Use `int` only for what you count or compare: ring count, max calls, timeouts.

Built-in data types



Type	Example	Collab use
`int`	`4`	ring count, max calls
`float`	`0.25`	packet loss ratio
`str`	`"1001"`	DN, device name, MAC
`bool`	`True`	phone registered?
`list`	`["1001", "1002"]`	lines on a phone
`tuple`	`("cucm-pub", 8443)`	host + port pair
`range`	`range(1001, 1100)`	generate a DN block
`dict`	`{"dn": "1001"}`	one AXL / REST record
`set`	`{"PT_Internal"}`	unique partitions
`bytes`	`b"INVITE sip:..."`	raw SIP message
`None`	`None`	empty field in a reply

Mutable vs immutable




```
dn = "1001"
before = id(dn)
dn += "0"
id(dn) == before        # False → new object

lines = ["1001"]
before = id(lines)
lines.append("1002")
id(lines) == before     # True → same object
```

IMMUTABLE

The value can never change after the object is created. Every "change" builds a **new** object and moves the label: `int`, `float`, `str`, `bool`, `tuple`, `range`, `bytes`, `None`.

MUTABLE

The object is changed **in place**. The `id` stays the same: `list`, `dict`, `set`, `bytearray`.

Why it matters:

- Two labels on a mutable box both see every change.
- Only immutable values can be `dict` keys or `set` items.

> ▸ **IN THE COLLAB WORLD**
> Reusing one phone template `dict` in a loop and editing it changes every phone built from it. Build a fresh `dict` per phone.

int · whole numbers

```
max_calls = 4
max_calls + 1           # 5
7 // 2, 7 % 2           # (3, 1)  floor div, remainder
7 / 2                   # 3.5  ← "/" always gives float
2 ** 100                # no overflow, any size
1_000_000               # underscores for readability

int("8443")             # 8443
int("1001A")            # ValueError
```

IMMUTABLE

`int` holds whole numbers, positive or negative, with no size limit. Use it for anything you **count** or **calculate**: calls, ports, timers, retries.

Arithmetic works as expected (`+ - * **`). Note that `/` always returns a `float`, while `//` keeps an `int`. `int()` converts text to a number and raises `ValueError` if the text is not a valid number.

▸ **IN THE COLLAB WORLD**

AXL returns every value as text: `"maxNumCalls": "4"`. Convert before comparing: `"10" > "9"` is `False` as strings, but `int("10") > int("9")` is `True`.

float · decimal numbers

```
lost, received = 12, 4800
loss = lost / received   # 0.0025
f"{loss:.2%}"            # '0.25%'

mos = float("4.1")       # text → float
mos >= 4.0               # True

0.1 + 0.2                # 0.30000000000000004
round(0.1 + 0.2, 2)      # 0.3
int(4.9)                 # 4  (cuts, no rounding)
```

IMMUTABLE

`float` holds numbers with a decimal point. Any division with `/` produces a float, even `4 / 2` gives `2.0`.

Floats are stored in binary, so some decimals cannot be stored exactly: `0.1 + 0.2` is not exactly `0.3`. Round when you **display** a value, and never compare two floats with `==`.

`int(x)` drops the decimal part. It does not round.

> ▸ **IN THE COLLAB WORLD**
> CMR (call management record) data gives packets lost and received. The loss ratio, jitter and MOS scores you calculate from them are `float`s.

Working with numbers

```
durations = [125, 340, 62, 410]  # seconds
sum(durations) / len(durations)  # 234.25
min(durations), max(durations)   # (62, 410)
divmod(3725, 60)        # (62, 5) → 62 min 5 s
round(234.567, 1)       # 234.6
round(2.5), round(3.5) # (2, 4) to even!
abs(-3)                 # 3
int("1F", 16), hex(31) # (31, '0x1f')
(4.0).is_integer()      # True
f"{1234567:,}"          # '1,234,567'
f"{0.0025:.2%}"         # '0.25%'
```

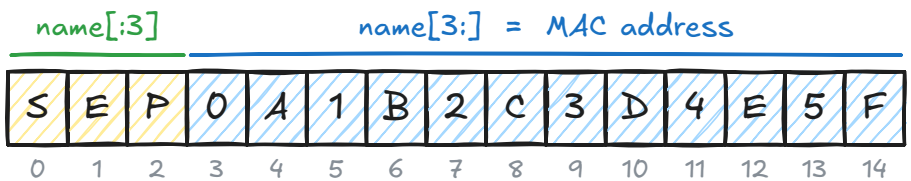
Numbers have few methods of their own. Most of the work is done by **built-in functions** and **f-string formats**:

Method	What it does
`sum` `min` `max`	total, smallest, largest of a list
`divmod(a, b)`	quotient and remainder in one call
`round(x, n)`	round to `n` digits (ties go to even)
`abs(x)`	distance from zero
`int(s, 16)` `hex(n)`	text in base 16 ↔ number
`float.is_integer()`	no decimal part?
`f"{x:.2f}"` `f"{x:,}"`	format for display only

▸ **IN THE COLLAB WORLD**

CDR durations are seconds. `divmod(duration, 60)` turns `3725` into 62 min 5 s for a readable call report.

str · text




```
name = "SEP0A1B2C3D4E5F"
len(name)        # 15
name[:3]         # 'SEP'
name[3:]         # '0A1B2C3D4E5F'
name.lower()     # new str, name unchanged
name[0] = "X"    # TypeError: immutable
```

IMMUTABLE

`str` is an ordered sequence of characters, written in `'single'`, `"double"` or `"""triple"""` quotes. Every character has an index starting at `0`, so you can read one character or slice a part.

Strings cannot be changed in place. Methods like `.lower()` or `.replace()` always return a **new** string. To keep the result, assign it: `name = name.lower()`.

▸ **IN THE COLLAB WORLD**

Most collab identifiers are strings: DNs, device names, MAC addresses, user IDs, SIP URIs. A phone's MAC is simply its name without the `SEP` prefix: `name[3:]`.

str methods · clean up




```
raw = "  sep0a1b2c3d4e5f \n"   # CSV cell
raw.strip()              # 'sep0a1b2c3d4e5f'
raw.strip().upper()      # 'SEP0A1B2C3D4E5F'
"alice smith".title()    # 'Alice Smith'
"0A:1B:2C:3D:4E:5F".replace(":", "")
                         # '0A1B2C3D4E5F'
"12".zfill(4)            # '0012'
"sip:1001".removeprefix("sip:")  # '1001'
```

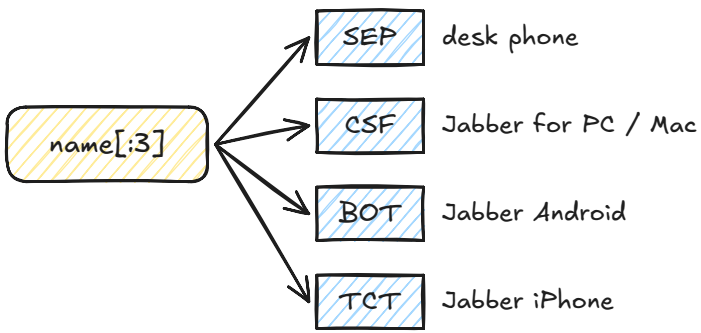
Clean-up methods return a **new** string, so they can be chained from left to right.

Method	What it does
`strip()`	remove spaces, tabs, newlines at both ends
`lstrip()` `rstrip()`	only left / only right side
`lower()` `upper()` `title()`	change letter case
`replace(old, new)`	swap every `old` with `new`
`zfill(n)`	pad with leading zeros to length `n`
`removeprefix(p)` `removesuffix(s)`	drop a fixed start / end

▸ **IN THE COLLAB WORLD**

MACs pasted from Excel come with spaces, lower case and colons. `"SEP" + mac.strip().upper().replace(":", "")` gives a valid device name for AXL.

str methods · search and test




```
name = "SEP0A1B2C3D4E5F"
name.startswith("SEP")      # True
"cucm-pub.clauto.lab".endswith(".lab")
                            # True
"A1" in name                # True
name.find("A1")             # 4  (-1 if missing)
name.index("ZZ")            # ValueError
"G711u,G711a,G722".count("G711")   # 2
"1001".isdigit()            # True
```

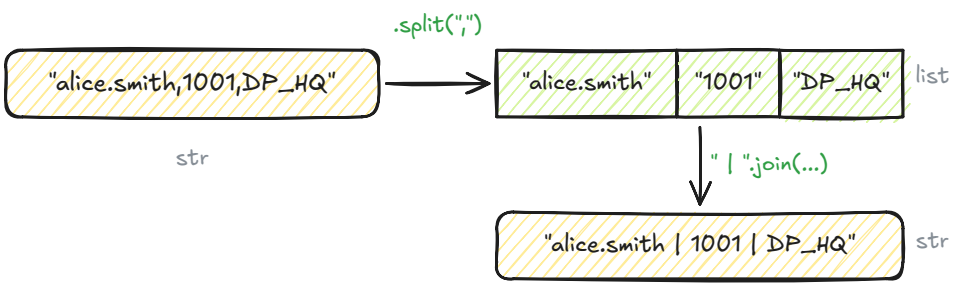
Search methods answer **where** or **whether** a piece of text appears. They never change the string.

Method	What it does
`startswith(p)` `endswith(s)`	does it begin / end with this?
`x in s`	does `x` appear anywhere? (operator)
`find(x)`	first position, or `-1` if not found
`index(x)`	like `find`, but raises `ValueError`
`count(x)`	how many times `x` appears
`isdigit()` `isalpha()` `isalnum()`	only digits / letters / both?

▸ IN THE COLLAB WORLD

The first three letters of a device name tell you the device type. `name.startswith("CSF")` finds every Jabber client in a phone list.

str methods · split, join, format




```
row = "alice.smith,1001,DP_HQ"
user, dn, pool = row.split(",")
uri = "sip:1001@cucm-pub.clauto.lab"
uri.split("@")     # ['sip:1001', 'cucm-pub…']
uri.partition("@") # ('sip:1001', '@', 'cucm…')
", ".join(["1001", "1002"])  # '1001, 1002'
f"{user:<12}{dn:>6}"  # 'alice.smith   1001'
```

`split` cuts a string into a **list**. `join` glues a list back into one string. They are opposites, and most parsing work is a mix of the two.

Method	What it does
`split(sep)`	cut at every `sep` → list (no `sep` = whitespace)
`split(sep, 1)` `rsplit(sep, 1)`	cut only once, from left / right
`partition(sep)`	cut once → `(before, sep, after)`
`sep.join(items)`	glue a list of strings with `sep`
`f"{x:<10}"` `f"{x:>6}"`	align left / right in a fixed width

▸ IN THE COLLAB WORLD

A SIP URI splits into user and host with `uri.partition("@")`. `join` builds the CSV lines and report rows you send back.

bool · True or False

```
status = "Registered"
is_reg = status == "Registered"   # True
type(is_reg)                      # <class 'bool'>

bool(""), bool("1001")    # (False, True)
bool([]), bool(["1001"])  # (False, True)
True + True               # 2  (bool is an int)

bool("false")             # True  ← careful!
```

IMMUTABLE

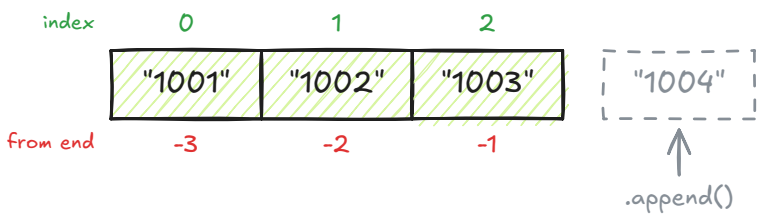
`bool` has exactly two values: `True` and `False`. Comparisons (`==`, `>`, `in`) produce a bool, and `if` statements consume them.

Every object has a **truth value**. Empty or zero values are **falsy**: `""`, `0`, `[]`, `{}`, `None`. Everything else is **truthy**. Under the hood, `bool` is a subtype of `int`: `True == 1`.

× COMMON MISTAKE

AXL sends booleans as text: `"true"` / `"false"`. Any non-empty string is truthy, so `bool("false")` is `True`. Compare instead: `enabled = value == "true"`.

list · ordered collection




```
lines = ["1001", "1002", "1003"]
lines[0]              # '1001'
lines[-1]             # '1003'
lines.append("1004")  # add to the end
lines[1] = "1099"     # change in place
len(lines)            # 4
```

MUTABLE

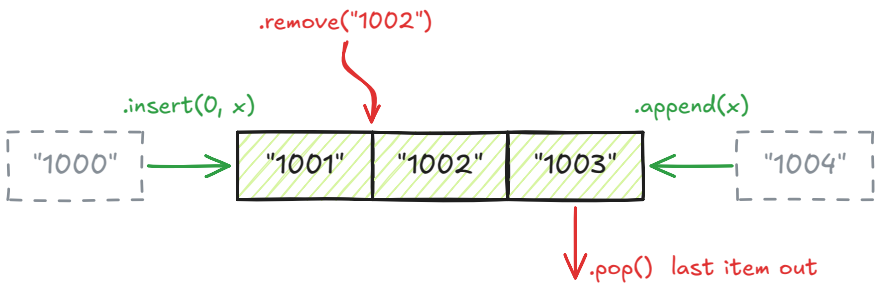
A `list` is an ordered collection in square brackets. It can hold any type, mixed types and duplicates. Items are read by index: `0` from the start, `-1` from the end.

Lists change in place: `append`, `insert`, `remove` and item assignment modify the same object. Their order is kept, so they are the natural container for “many of something”.

▸ **IN THE COLLAB WORLD**

`listPhone` returns a list of phones, a CSV import is a list of rows, and a phone’s lines are a list of DNs. Most automation scripts loop over a list.

list methods · add and remove




```
lines = ["1001", "1002"]
lines.append("1003")          # one, at the end
lines.extend(["1004", "1005"])  # many
lines.insert(0, "1000")       # at index 0
lines.remove("1002")          # by value
lines.pop()                   # '1005' last
lines.pop(0)                  # '1000' by index
lines         # ['1001', '1003', '1004']
```

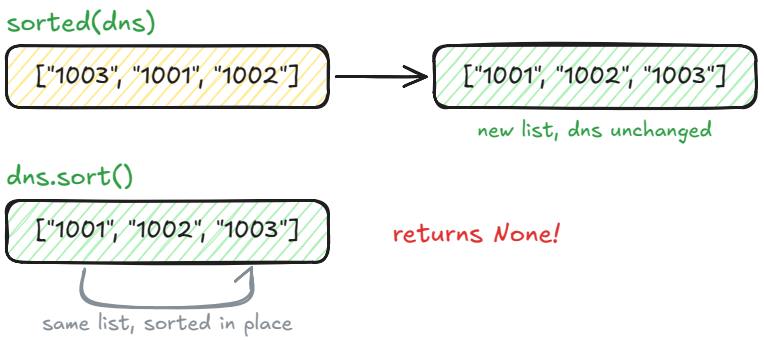
These methods change the list **in place** and keep the same `id`.

Method	What it does
`append(x)`	add one item to the end
`extend(items)`	add every item of another list
`insert(i, x)`	add `x` before position `i`
`remove(x)`	delete the first `x` (`ValueError` if missing)
`pop()` `pop(i)`	remove and **return** last / item `i`
`clear()`	remove everything

× COMMON MISTAKE

`lines.append(["1002", "1003"])` adds **one** item, a nested list: `["1001", ["1002", "1003"]]`. Use `extend` to add many.

list methods · order and search




```
dns = ["1003", "1001", "1002"]
sorted(dns)           # new sorted list
dns.sort(reverse=True)  # in place
dns.index("1002")     # 1
dns.count("1001")     # 1
"1004" in dns         # False
backup = dns.copy()   # a real copy
sorted(names, key=str.lower)  # ignore case
```

Ordering and searching work on any list. Note the two ways to sort.

Method	What it does
`sorted(lst)`	**new** sorted list, original untouched
`lst.sort()`	sort in place, returns `None`
`reverse=True` `key=...`	descending order / sort by a rule
`reverse()`	flip the order in place
`index(x)` `count(x)`	position of `x` / how many `x`
`copy()`	shallow copy: a new list, same items

× COMMON MISTAKE

`dns = dns.sort()` sets `dns` to `None`, because `sort()` returns nothing. Use `dns.sort()` alone, or `dns = sorted(dns)`.

tuple · fixed sequence

```
server = ("cucm-pub.clauto.lab", 8443)
server[0]             # 'cucm-pub.clauto.lab'
host, port = server   # unpacking
server[1] = 443       # TypeError: immutable

codecs = ("G722", "G711u", "G711a")
"G722" in codecs      # True

nodes = {("cucm-pub.clauto.lab", 8443): "pub"}
```

IMMUTABLE

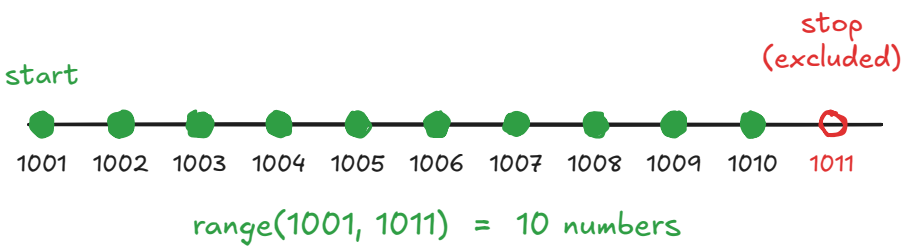
A `tuple` looks like a list in round brackets, but it cannot change after creation. Use it for a fixed group of values that belong together, like a host and its port.

Unpacking assigns each item to its own name in one line. Because tuples are immutable, they can be `dict` keys and `set` items. Lists cannot.

▸ **IN THE COLLAB WORLD**

A server address `(host, port)`, a fixed codec preference order, or a function result `(ok, message)` after an AXL call are all good tuples.

range · number sequence




```
block = range(1001, 1011)
len(block)                  # 10
1005 in block               # True
list(range(1001, 1011, 3)) # [1001, 1004, 1007, 1010]

for n in range(1001, 1004):
    print("DN", str(n))     # DN 1001 … DN 1003
```

IMMUTABLE

`range(start, stop, step)` describes a sequence of whole numbers. `start` is included and `stop` is **not**. The default step is `1`.

A range does not store its numbers. It calculates them when asked, so `range(1, 1_000_000)` uses almost no memory. Wrap it in `list()` to see the values.

▸ **IN THE COLLAB WORLD**

Generate a DN block for a new site with `range(1001, 1100)`. Remember that DNs are strings: convert each number with `str(n)` before sending it to AXL.

tuple and range · useful operations

```
codecs = ("G722", "G711u", "G711a", "G711u")
codecs.count("G711u")     # 2
codecs.index("G711a")     # 2
len(codecs), codecs[:2]   # (4, ('G722', 'G711u'))

block = range(1001, 1100)
block.start, block.stop   # (1001, 1100)
block[-1], len(block)     # (1099, 99)
list(range(1003, 1000, -1))   # [1003, 1002, 1001]

dns = ["1001", "1002"]
users = ["alice.smith", "bob.jones"]
dict(zip(dns, users))     # {'1001': 'alice.smith', …}
list(enumerate(dns, start=1))  # [(1, '1001'), …]
```

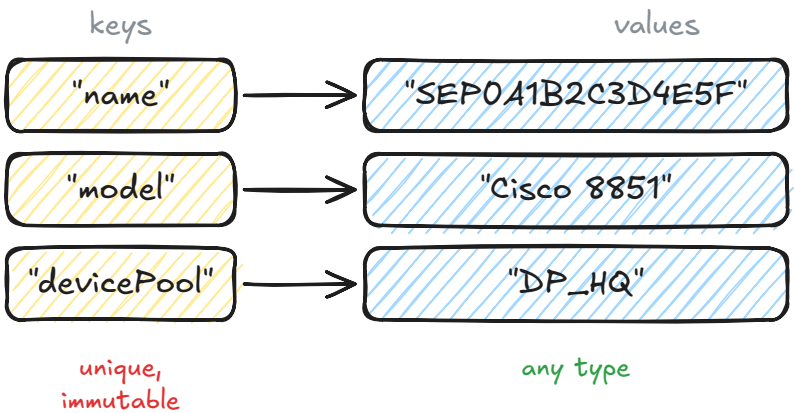
A `tuple` has only two methods. Everything else comes from operations every sequence shares: `len`, `in`, indexing, slicing, unpacking. A `range` supports the same, plus its `start`, `stop` and `step`.

Method	What it does
`count(x)` `index(x)`	how many `x` / position of `x`
`len` `in` `[i]` `[a:b]`	size, membership, index, slice
`range(a, b, -1)`	count down with a negative step
`zip(a, b)`	pair items of two sequences
`enumerate(seq, start=1)`	number each item

▸ **IN THE COLLAB WORLD**

Two Excel columns, DNs and user IDs, become one lookup table with `dict(zip(dns, users))`.

dict · key → value




```
phone = {"name": "SEP0A1B2C3D4E5F",
         "model": "Cisco 8851"}
phone["model"]                  # 'Cisco 8851'
phone["devicePool"] = "DP_HQ"   # add a key
phone.get("css")                # None, no error
"name" in phone                 # True
```

MUTABLE

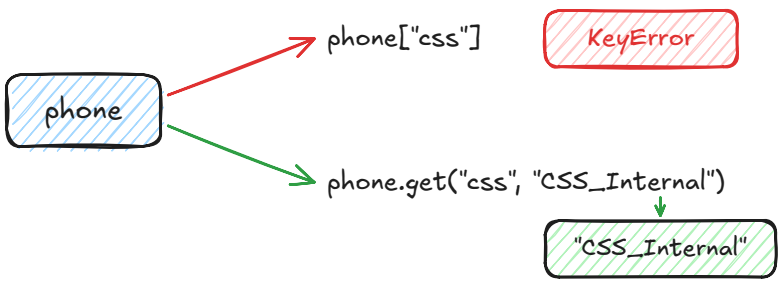
A `dict` maps **keys** to **values** in curly braces. Keys are unique and must be immutable (usually `str`). Values can be anything, even other dicts or lists. Python keeps the insertion order.

Read with `phone["model"]`. A missing key raises `KeyError`. `phone.get("css")` returns `None` instead. Adding or changing a key modifies the dict in place.

▸ **IN THE COLLAB WORLD**

Every AXL or REST record is a dict: a phone, a line, a Webex user. JSON from the Webex API maps directly to nested dicts and lists.

dict methods · read and update




```
phone = {"name": "SEP0A1B2C3D4E5F",
         "model": "Cisco 8851"}
phone.get("css")                    # None
phone.get("css", "CSS_Internal")  # default
phone.update({"devicePool": "DP_HQ",
              "description": "Alice desk"})
phone.pop("description")    # 'Alice desk'
phone.setdefault("css", "CSS_Internal")
del phone["css"]
```

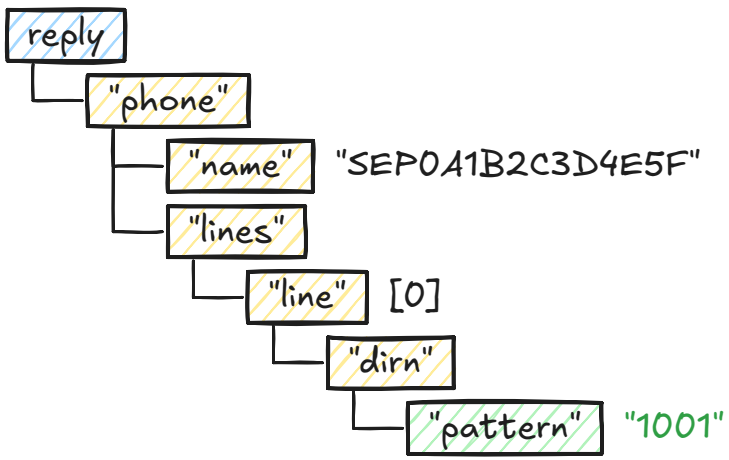
`[]` is strict: a missing key is an error. `get` is forgiving: it returns `None` or the default you give.

Method	What it does
`d[k]`	value of `k`, `KeyError` if missing
`get(k, default)`	value of `k`, or `default` / `None`
`update(other)`	add or overwrite many keys at once
`pop(k)`	remove `k` and return its value
`setdefault(k, v)`	set `k` to `v` only if it is missing
`del d[k]`	remove `k` (statement, not a method)

▸ IN THE COLLAB WORLD

Optional AXL fields may be missing from a reply. Read them with `phone.get("css")` so one incomplete record does not stop a script that walks 500 phones.

dict methods · loop and nest




```python
for key, value in phone.items():
    print(f"{key:<12} {value}")

list(phone.keys())     # ['name', 'model']
"SEP0A1B2C3D4E5F" in phone.values()  # True

line = reply["phone"]["lines"]["line"][0]
line["dirn"]["pattern"]   # '1001'
copy = phone.copy()       # a new dict
```

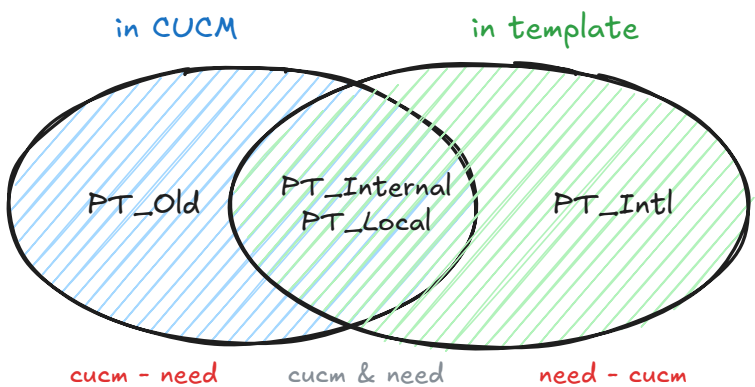
Three **views** expose a dict's content for loops and tests. Nested dicts and lists are read one level at a time.

Method	What it does
`keys()`	all keys
`values()`	all values
`items()`	`(key, value)` pairs, the usual loop
`k in d`	is `k` a key? (checks keys, not values)
`d[a][b][0]`	step down into nested data
`copy()`	shallow copy: new dict, same inner objects

▸ **IN THE COLLAB WORLD**

`getPhone` returns deeply nested data. Store an inner level in a variable (`line = …`) to keep each step short and readable.

set · unique items




```
cucm = {"PT_Internal", "PT_Local", "PT_Old"}
need = {"PT_Internal", "PT_Local", "PT_Intl"}
need - cucm    # {'PT_Intl'}  missing
cucm - need    # {'PT_Old'}   unused
cucm & need    # in both
set(["1001", "1001", "1002"])  # duplicates gone
```

MUTABLE

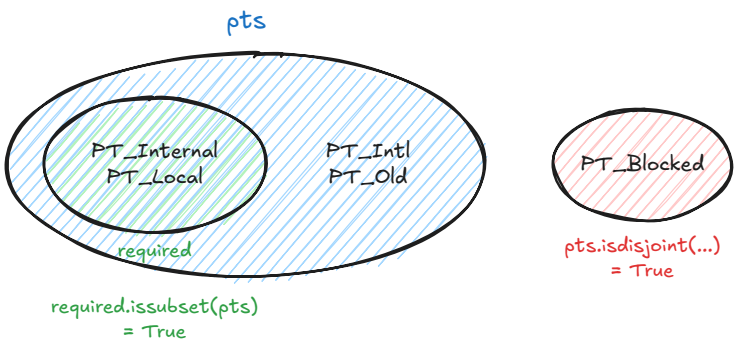
A `set` is an unordered collection of **unique**, immutable items. Adding a value that is already there does nothing. A set has no index and no guaranteed order.

Sets answer two questions very fast: “is this item in it?” (`in`) and “how do two groups differ?” Use `-` for difference, `&` for intersection and `|` for union.

▸ **IN THE COLLAB WORLD**

Config audits: compare the partitions in CUCM with the partitions in your design document. The missing and unused ones come out in one line.

set methods




```
pts = {"PT_Internal"}
pts.add("PT_Local")        # add one
pts.add("PT_Local")        # no effect, exists
pts.update(["PT_Intl", "PT_Old"])  # add many
pts.discard("PT_None")     # missing: no error
pts.remove("PT_None")      # missing: KeyError

required = {"PT_Internal", "PT_Local"}
required.issubset(pts)     # True
pts.isdisjoint({"PT_Blocked"})  # True
```

Set methods add and remove items, and compare whole groups. The operators `| & -` from the previous slide have method versions that also accept lists.

Method	What it does
`add(x)` `update(items)`	add one / many (duplicates ignored)
`discard(x)` `remove(x)`	delete, silently / with `KeyError`
`union` `intersection` `difference`	same as `|`, `&`, `-`
`issubset(b)` `issuperset(b)`	all inside `b` / contains all of `b`
`isdisjoint(b)`	no item in common?

▸ **IN THE COLLAB WORLD**

Check that a CSS contains every required partition with `required.issubset(css_partitions)`: one line, one `True` or `False`.

bytes · raw binary data

```
msg = "INVITE sip:1001@cucm-pub.clauto.lab SIP/2.0"
raw = msg.encode()      # str → bytes (UTF-8)
raw[:6]                 # b'INVITE'
raw[0]                  # 73  ← one byte is a number
raw.decode()            # bytes → str

buf = bytearray(raw)    # mutable copy
buf[0:6] = b"CANCEL"    # change in place
```

IMMUTABLE `bytes` MUTABLE `bytearray`

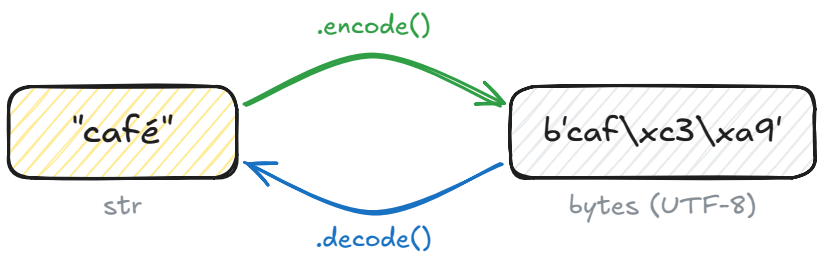
`bytes` is a sequence of raw bytes (values 0–255), written as `b"..."`. Text becomes bytes with `.encode()` and back with `.decode()`. `bytearray` is the mutable version.

Networks and files move bytes, not text. You meet them when reading sockets, files and certificates.

▸ **IN THE COLLAB WORLD**

With `requests`, `r.text` is a `str` and `r.content` is `bytes`. Use `bytes` for downloads like CDR files or certificates, and for raw SIP messages.

bytes methods




```
raw = b"SIP/2.0 200 OK\r\nCSeq: 1 INVITE\r\n"
raw.startswith(b"SIP/2.0 200")  # True
raw.split(b"\r\n")[0].decode()  # 'SIP/2.0 200 OK'
raw.find(b"CSeq")               # 16
raw.hex()[:8]                   # '5349502f'
bytes.fromhex("53495020")       # b'SIP '
"café".encode()                 # b'caf\xc3\xa9'
b"caf\xe9".decode(errors="replace")  # 'caf\ufffd'
```

`bytes` shares most `str` methods (`split`, `find`, `startswith`, `strip`), but every argument must be `bytes` too: `b"..."`. Converting between the two is the key step.

Method	What it does
`s.encode()`	`str` → `bytes` (UTF-8 by default)
`b.decode()`	`bytes` → `str`
`decode(errors="replace")`	do not crash on bad bytes
`hex()` `bytes.fromhex(s)`	bytes ↔ hex text
`split` `find` `startswith`	same as `str`, with `b"..."`

▸ IN THE COLLAB WORLD

AXL uses HTTP Basic auth: the header is `base64.b64encode(b"axl_svc:password")`. `requests` does this for you, but it is bytes underneath.

None · no value

```
phone = {"name": "SEP0A1B2C3D4E5F",
         "description": None}
desc = phone["description"]
desc is None                # True
phone.get("css")            # None, key missing

result = print("hi")        # print returns None
desc or "no description"    # fallback value
desc.upper()                # AttributeError
```

IMMUTABLE

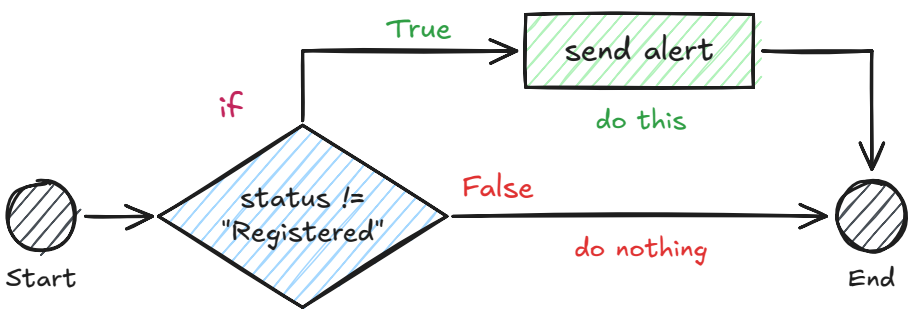
`None` means “no value here”. It is the only object of type `NoneType`. A function without a `return` returns `None`, and `dict.get()` returns `None` for a missing key.

`None` is falsy. Check for it with `is None`, not `== None`. Use `x or default` to fall back to a value.

× COMMON MISTAKE

Empty AXL fields come back as `None`. Calling a string method on it, like `desc.upper()`, raises an `AttributeError`. Check for `None` first, or use a fallback: `(desc or "").upper()`.

if · run code only when




```
status = "UnRegistered"

if status != "Registered":
    print("ALERT: SEP0A1B2C3D4E5F is down")

print("check finished")   # always runs
```

`if` runs a block of code only when its **condition** is `True`. When the condition is `False`, Python skips the block and continues.

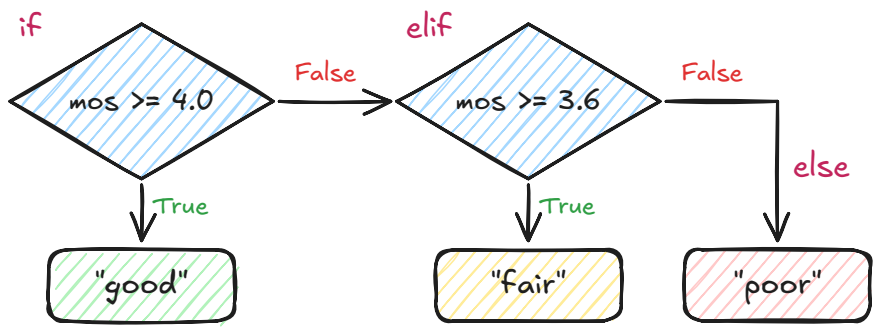
The line ends with a colon `:`. The block below is **indented** with 4 spaces; indentation, not brackets, tells Python where the block ends.

Operator	True when
`==` `!=`	equal / not equal
`<` `>` `<=` `>=`	smaller / greater (or equal)
`in` `not in`	item is / is not inside a sequence
`is` `is not`	same object, e.g. `x is None`

▸ **IN THE COLLAB WORLD**

RISPort returns a status per device. `if status != "Registered"` is the heart of every phone-down alert.

if / elif / else · choose one path



checked top to bottom, only the first True branch runs

```
mos = 3.8

if mos >= 4.0:
    quality = "good"
elif mos >= 3.6:
    quality = "fair"
else:
    quality = "poor"

print(f"MOS {mos}: {quality}")  # MOS 3.8: fair
```

`elif` ("else if") adds more conditions, as many as you need. `else` catches everything that matched none of them. Both are optional.

Python checks the conditions **from top to bottom** and runs only the first branch that is `True`. All other branches are skipped, even if they would also be `True`.

× COMMON MISTAKE

Order matters. If `mos >= 3.6` comes first, a MOS of `4.5` is labelled `"fair"`, because that branch matches first. Put the strictest condition at the top.

▸ IN THE COLLAB WORLD

Grade call quality from CMR data: MOS 4.0 and above is good, 3.6 to 4.0 is fair, below 3.6 needs investigation.

Combining conditions

```
status = "UnRegistered"
model = "Cisco 8851"
lines = []                          # no DN yet

if status != "Registered" and "88" in model:
    print("8800 series phone is down")

if not lines:                       # empty → falsy
    print("phone has no line")

if model in ("Cisco 8851", "Cisco 8865"):
    print("video capable")

label = "OK" if status == "Registered" else "DOWN"
```

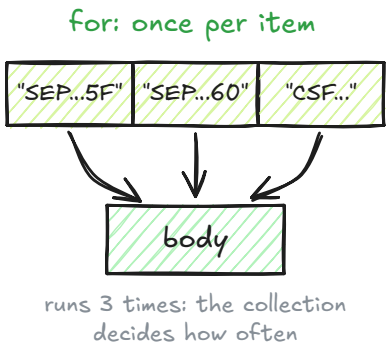
Conditions can be combined, and any value can be a condition: empty values (`""`, `[]`, `{}`, `0`, `None`) count as `False`.

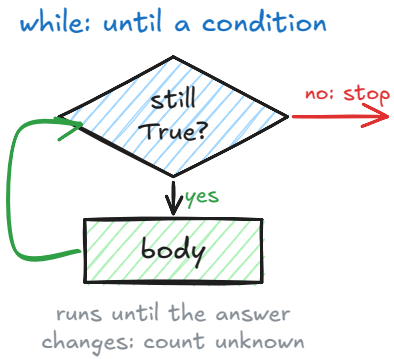
Operator	True when
`a and b`	both are true
`a or b`	at least one is true
`not a`	`a` is false
`x if c else y`	one-line choice: `x` when `c`, else `y`

× COMMON MISTAKE

`if model == "Cisco 8851" or "Cisco 8865":` is **always** `True`: the non-empty string `"Cisco 8865"` is truthy on its own. Write `if model in ("Cisco 8851", "Cisco 8865"):`.

What is a loop?






```
# without a loop: one line per phone
print("reset", "SEP0A1B2C3D4E5F")
print("reset", "SEP0A1B2C3D4E60")
# ... 498 more lines

# with a loop: two lines for any number
for name in phones:
    print("reset", name)
```

A **loop** repeats a block of code. You write the work once and Python runs it as many times as needed: for 3 phones or for 3000. This is where automation saves its time.

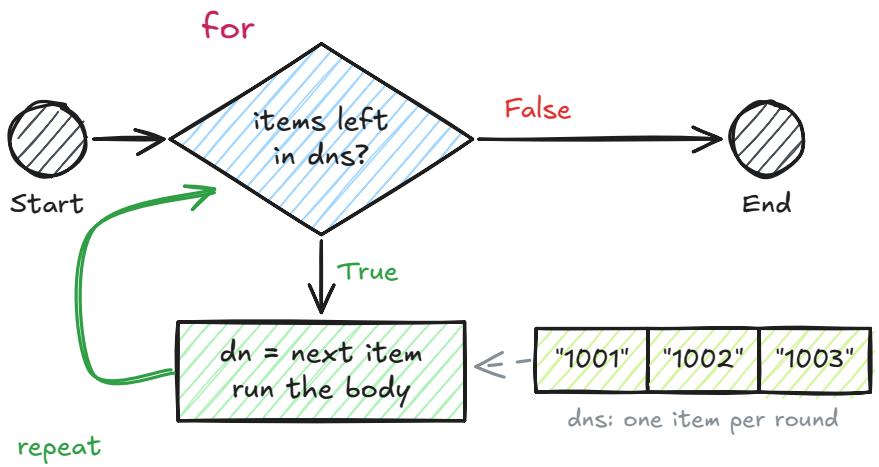
Term	Meaning
body	the indented block that is repeated
iteration	one run of the body (one “round”)
loop variable	name that holds the current item
condition	`True` / `False` test that decides to go on

Python has two loops. `for` repeats **once per item** of a collection. `while` repeats **as long as a condition** stays `True`.

▸ **IN THE COLLAB WORLD**

Reset, audit or update 500 phones: the code for one phone goes in the body, the loop takes care of the other 499.

for · repeat for every item




```
dns = ["1001", "1002", "1003"]

for dn in dns:
    print(f"Adding line {dn}")

print("done")          # after the loop
```

A `for` loop runs its body **once for every item** of a collection. In each round, the loop variable (`dn`) holds the next item. When no items are left, the loop ends and the code after it runs.

Like `if`, the line ends with `:` and the body is indented. The loop variable is a normal name: pick one that says what one item is.

`for` works on anything you can step through: `list`, `tuple`, `str`, `dict`, `set`, `range`, and later files and API replies.

▸ **IN THE COLLAB WORLD**

This is the core of bulk provisioning: read 500 rows from a CSV, then `for row in rows:` build and send one AXL `addLine` or `addPhone` request per row.

for · common patterns

```python
for n in range(1001, 1004):              # numbers
    print(str(n))

for i, dn in enumerate(dns, start=1):    # position + item
    print(i, dn)                         # 1 1001 …

for key, value in phone.items():         # dict pairs
    print(key, value)

for dn, user in zip(dns, users):         # two lists at once
    print(dn, "->", user)                # 1001 -> alice.smith

total = 0
for d in durations:                      # accumulate
    total += d                           # 937
```

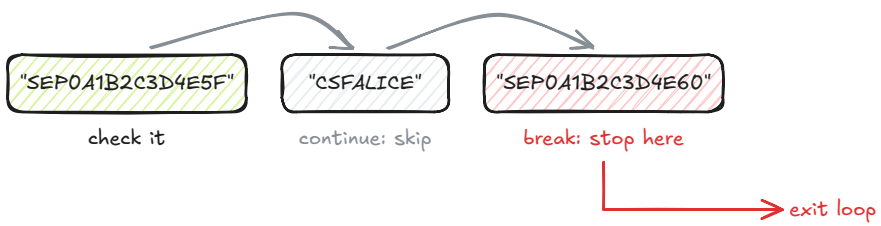
Five patterns cover most loops in automation scripts:

Pattern	Use it to
`range(a, b)`	repeat for a block of numbers
`enumerate(seq)`	get the position and the item
`d.items()`	walk a dict key by key
`zip(a, b)`	walk two lists side by side
`total += x`	build a sum, count or new list

▸ **IN THE COLLAB WORLD**

`zip(dns, users)` pairs the DN column with the user column of an Excel sheet, ready to associate each line with its owner.

break and continue




```
phones = ["SEP0A1B2C3D4E5F", "CSFALICE",
          "SEP0A1B2C3D4E60"]

for name in phones:
    if name.startswith("CSF"):
        continue          # skip, go to next item
    if name == "SEP0A1B2C3D4E60":
        print("found", name)
        break             # leave the loop now
    print("checking", name)
```

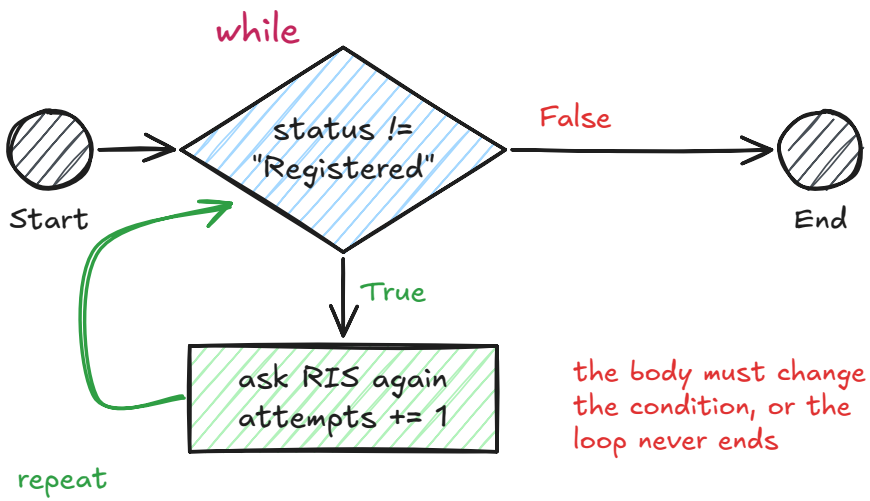
Two keywords change the normal flow of a loop:

- `continue` skips the rest of the body and starts the **next round**.
- `break` leaves the loop **immediately**. The remaining items are not visited.

Both are usually inside an `if`. They work the same way in `for` and `while` loops.

> ▸ **IN THE COLLAB WORLD**
> `continue` to skip Jabber clients (`CSF…`) in a desk phone audit.
> `break` as soon as the device you are searching for is found, so the script does not walk the remaining thousand phones.

while · repeat while true




```
replies = ["Unknown", "Unknown", "Registered"]
attempts = 0
status = ""

while status != "Registered" and attempts < 5:
    status = replies[attempts]   # RIS reply
    attempts += 1
    print(f"try {attempts}: {status}")
```

A `while` loop repeats its body **as long as** its condition is `True`. The condition is checked before every round. As soon as it is `False`, the loop ends.

Use `while` when you do not know in advance how many rounds you need: “until the phone is registered”, “until the job is done”.

> **× COMMON MISTAKE**
> If nothing in the body changes the condition, the loop never ends. Always add a limit like `attempts < 5`. Stop a stuck script with `Ctrl+C`.

while True · poll until done

```python
import time

attempts = 0
while True:
    attempts += 1
    status = get_status("SEP0A1B2C3D4E5F")  # your RIS call
    if status == "Registered":
        print("phone is back")
        break
    if attempts == 10:
        print("giving up after 10 tries")
        break
    time.sleep(30)          # wait before asking again
```

`while True` loops forever on purpose. The exits are written inside the body with `break`. This keeps several exit rules readable.

Loop	Best when
`for`	you have a collection or a known count
`while cond`	repeat until a condition changes
`while True` + `break`	several exit rules, polling

▸ IN THE COLLAB WORLD

After resetting a phone, poll RIS until it registers again. RIS is rate-limited, so always `sleep` between requests and cap the number of attempts.

What is a function?




```python
def build_device_name(mac):
    """Turn a raw MAC into a CUCM device name."""
    clean = mac.strip().upper().replace(":", "")
    return "SEP" + clean

name = build_device_name("0a:1b:2c:3d:4e:5f")
print(name)        # SEP0A1B2C3D4E5F
```

A **function** is a named block of code that does one job. You **define** it once with `def`, then **call** it by name as often as you need, with different input each time.

Part	Meaning
`def name(...):`	define the function, body indented below
parameter	the input name in the definition (`mac`)
argument	the actual value passed in a call
`return`	send a result back and leave the function
`"""docstring"""`	one line that says what it does

▸ **IN THE COLLAB WORLD**

Every rule you apply more than once (naming, cleaning, validating) belongs in a function: fix it in one place, every script benefits.

Parameters, defaults and return

```python
def line_label(dn, user, site="HQ"):
    return f"{user} ({dn}) - {site}"

line_label("1001", "alice.smith")    # alice.smith (1001) - HQ
line_label("1002", "bob.jones", "Branch")      # positional
line_label(user="carol.white", dn="1003")      # keyword

def check(status):
    if status == "Registered":
        return True, "ok"
    return False, f"status is {status}"

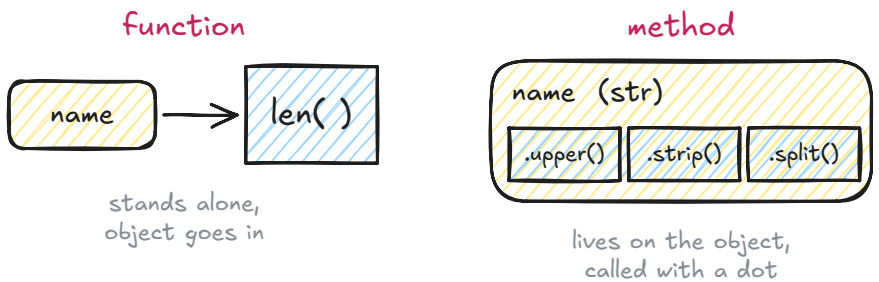
ok, msg = check("UnRegistered")    # two values back (tuple)
```

- **Positional** arguments match parameters by order, **keyword** arguments by name (`user="…"`), in any order.
- A **default** value (`site="HQ"`) makes a parameter optional.
- `return a, b` returns a tuple. Without `return`, a function returns `None`.
- Names created inside a function are **local**: they disappear when the function ends.

× COMMON MISTAKE

Never use a mutable default: `def add(dn, lines=[])` shares **one** list between all calls. Use `lines=None`, then `if lines is None: lines = []`.

Function vs method




```
name = "sep0a1b2c3d4e5f"

len(name)               # function: object goes in
name.upper()            # method: called on the object
str.upper(name)         # same method, function style

lines = ["1001"]
lines.append("1002")    # list method
len(lines)              # one len() for every type

name.append("x")        # AttributeError: no such
                        # method on str
```

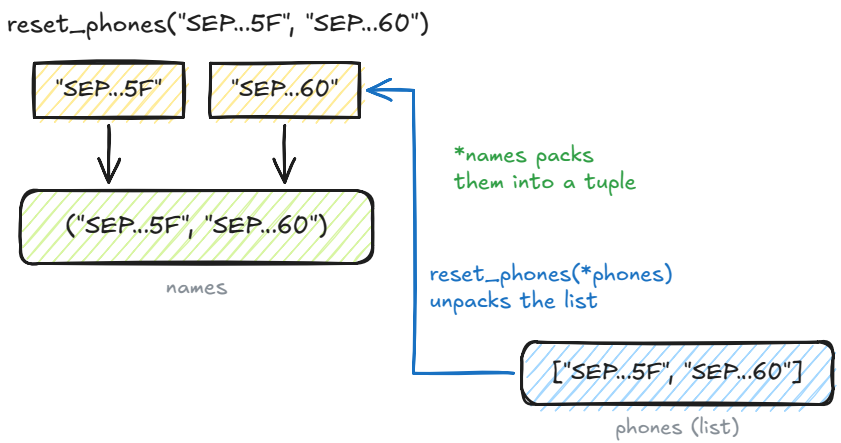
A **method** is a function that **belongs to an object's type**. You call it with a dot on the object, and the object is passed in automatically. A plain **function** stands alone and gets the object as an argument.

Function	Method
`len(name)`	`name.upper()`
defined with `def` in a module	defined with `def` inside a class
works on many types	only exists on its own type
object passed as argument	object passed automatically (`self`)

! WATCH OUT

Not every dot is a method. In `requests.get(url)` or `json.loads(text)` the dot only opens a **module**: `get` and `loads` are plain functions.

*args · any number of positional arguments




```python
def reset_phones(*names):
    print(type(names))      # <class 'tuple'>
    for name in names:
        print("reset", name)

reset_phones("SEP0A1B2C3D4E5F")
reset_phones("SEP0A1B2C3D4E5F", "SEP0A1B2C3D4E60")
reset_phones(*phones)       # unpack a list
```

A parameter with a single star collects **all extra positional arguments** into a `tuple`. The function can be called with zero, one or a hundred values.

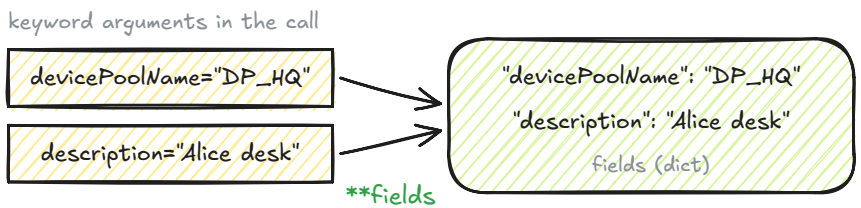
The name after `*` is free; `*args` is the convention when there is no better name. Normal parameters can come first: `def reset(site, *names)`.

In a **call**, the star works the other way round: `f(*phones)` unpacks a list into separate arguments.

▸ **IN THE COLLAB WORLD**

One `reset_phones()` helper serves a single phone from the CLI and a whole list from a CSV import, without two versions of the function.

**kwargs · any number of keyword arguments




```python
def add_phone(name, model, **fields):
    phone = {"name": name, "model": model}
    phone.update(fields)       # fields is a dict
    return phone

add_phone("SEP0A1B2C3D4E5F", "Cisco 8851",
          devicePoolName="DP_HQ",
          description="Alice desk")

defaults = {"devicePoolName": "DP_HQ"}
add_phone("SEP0A1B2C3D4E60", "Cisco 8865",
          **defaults)          # unpack a dict
```

A parameter with two stars collects **all extra keyword arguments** into a `dict`: the names become keys, the values become values.

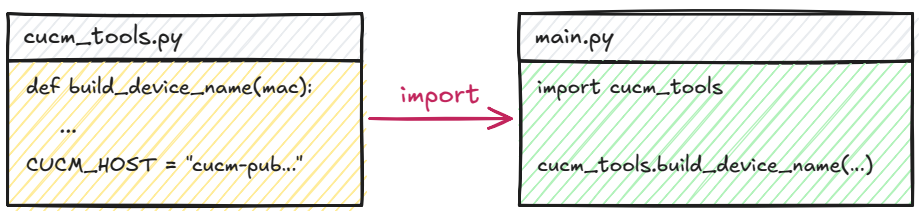
Syntax	Meaning
`def f(*args)`	extra positional → `tuple`
`def f(**kwargs)`	extra keyword → `dict`
`f(*lst)`	unpack a list into positional args
`f(**d)`	unpack a dict into keyword args

Order in a definition: normal, `*args`, keyword-only, `**kwargs`.

▸ **IN THE COLLAB WORLD**

An AXL phone has dozens of optional fields. With `**fields` one helper accepts any of them without listing every one.

Modules · use code from another file



same folder: Python finds the file by its name

```
# main.py
import cucm_tools
cucm_tools.build_device_name("0a:1b:2c:3d:4e:5f")

from cucm_tools import build_device_name, CUCM_HOST
build_device_name("0a:1b:2c:3d:4e:60")

import cucm_tools as ct      # short alias
ct.CUCM_HOST                 # 'cucm-pub.clauto.lab'
```

Every `.py` file is a **module**. `import` loads another file once and gives you access to its functions and variables, so code is written in one place and used everywhere.

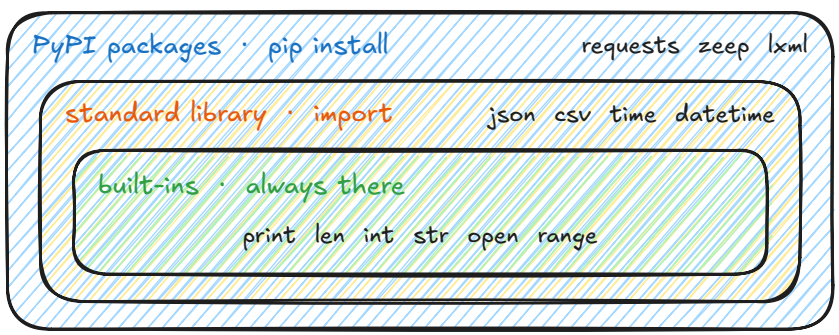
Form	Then use
`import m`	`m.func()`: clear where it comes from
`from m import func`	`func()`: shorter, pick names
`import m as x`	`x.func()`: alias for long names

Python looks for modules in the script's folder, then in the standard library, then in installed packages.

× COMMON MISTAKE

Never name your own file like a real module: a `json.py` or `requests.py` in your folder hides the real one.

Standard library and packages




```python
import json, time                  # standard library
from datetime import datetime

json.loads('{"dn": "1001"}')       # str → dict
time.sleep(2)                      # wait 2 seconds
datetime.now().strftime("%Y-%m-%d")

import requests                    # pip install requests
```

Three layers of ready-made code:

- **Built-ins** like `print` and `len` need no import.
- The **standard library** ships with Python: `import json`.
- **Third-party packages** come from PyPI: `pip install requests`, then `import requests`.

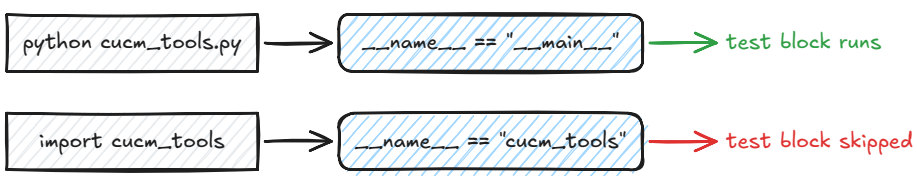
Install packages into a **virtual environment** per project, so every project keeps its own versions:

```
python3 -m venv .venv && source .venv/bin/activate
pip install requests zeep
```

▸ **IN THE COLLAB WORLD**

`requests` for REST (Webex, CUPI), `zeep` for SOAP (AXL, RIS), `json` and `csv` for the data around them.

Scripts vs modules · __name__




```python
# cucm_tools.py
def build_device_name(mac):
    return "SEP" + mac.replace(":", "").upper()

if __name__ == "__main__":
    # only when run directly
    print(build_device_name("0a:1b:2c:3d:4e:5f"))
```

Python sets `__name__` to `"__main__"` in the file you run, and to the module name in a file that is imported. The `if` block is a self-test that never runs on import.

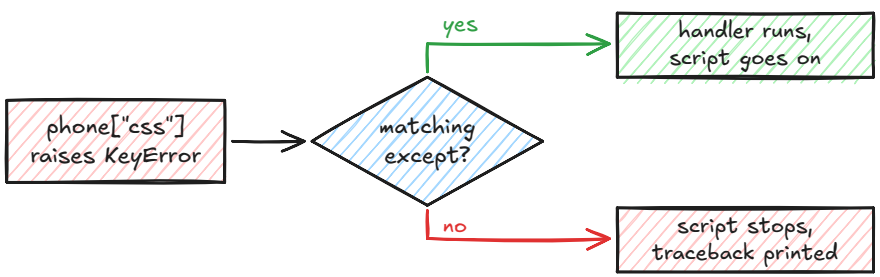
A folder of modules is a **package**. Import with dots:

```
clauto/
├── main.py          from cucm.axl import add_phone
└── cucm/
    ├── __init__.py  (marks the folder as a package)
    ├── axl.py
    └── ris.py
```

▸ IN THE COLLAB WORLD

Keep AXL, RIS and Webex helpers in their own modules. Every new script then starts with two `import` lines instead of copy-paste.

What is an exception?




```
phone = {"name": "SEP0A1B2C3D4E5F"}
phone["css"]
```

```
Traceback (most recent call last):
  File "audit.py", line 2, in <module>
    phone["css"]
KeyError: 'css'
```

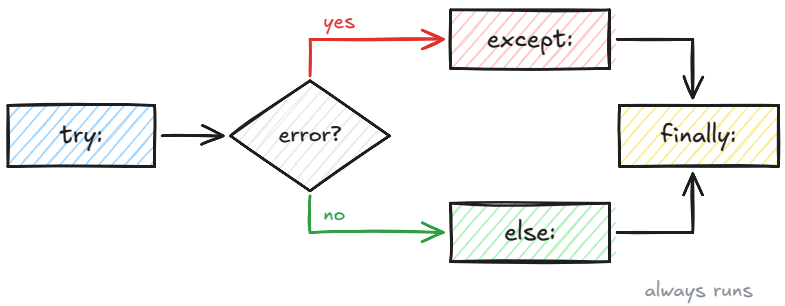
An **exception** is Python's way of saying "I cannot continue here". If nothing handles it, the script stops and prints a **traceback**. Read it from the **bottom**: the last line names the error type and the message, the lines above show where it happened.

Exception	Typical cause
`KeyError`	missing key in a dict
`IndexError`	list position does not exist
`ValueError`	right type, wrong value: `int("84x3")`
`TypeError`	wrong type: `"1001" + 1`
`AttributeError`	no such method: `None.upper()`
`FileNotFoundError`	CSV path is wrong

▸ **IN THE COLLAB WORLD**

Libraries add their own: `requests` raises `Timeout`, `zeep` raises a `Fault` when AXL rejects a request (duplicate name, missing CSS).

try / except / else / finally




```python
def to_int(text):
    try:
        value = int(text)
    except ValueError:
        print(f"not a number: {text!r}")
        value = None
    else:
        print("converted", value)
    finally:
        print("checked", text)
    return value

to_int("8443")    # converted 8443, checked 8443
to_int("84x3")    # not a number: '84x3', checked 84x3
```

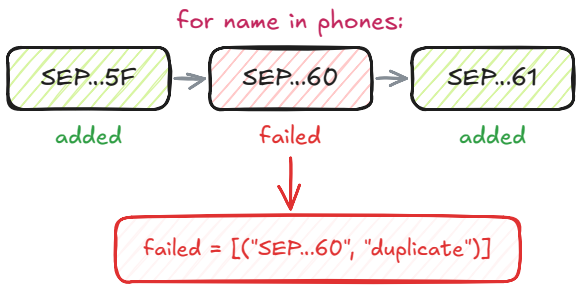
Put risky code in `try`. If an exception of the named type is raised, Python jumps to the matching `except` block instead of stopping.

Block	Runs when
`try:`	always first: the risky code
`except Error:`	that error happened in `try`
`except (A, B) as e:`	one of several, `e` is the error
`else:`	`try` finished **without** an error
`finally:`	always, error or not: clean-up

× COMMON MISTAKE

A bare `except:` or `except Exception: pass` hides every bug, typos included. Catch the specific error you expect, and always print or log what went wrong.

Errors in bulk jobs · raise




```python
failed = []
for name in phones:
    try:
        add_phone_axl(name)      # your AXL call
    except Exception as e:
        failed.append((name, str(e)))
        continue
    print("added", name)
print(f"{len(failed)} failed")

def build_device_name(mac):
    mac = mac.replace(":", "").upper()
    if len(mac) != 12:
        raise ValueError(f"bad MAC: {mac!r}")
    return "SEP" + mac
```

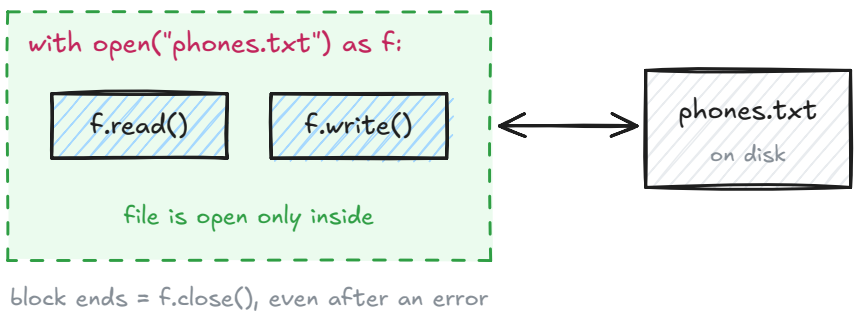
In a bulk job one bad row must not stop the others. Put `try` **inside** the loop: record the failure, `continue` with the next item, and report all failures at the end. Here a broad `except Exception` is fine, because the error is stored and shown.

`raise` creates an exception yourself. Use it when input is wrong, **before** it reaches CUCM: a clear `ValueError("bad MAC")` is much easier to debug than a confusing AXL fault later.

▸ **IN THE COLLAB WORLD**

Import 500 phones: 499 are added, 1 duplicate is written to `failed.csv` with its AXL error message for a second run.

Working with files




```
f = open("phones.txt", encoding="utf-8")
text = f.read()
f.close()                    # easy to forget

with open("phones.txt", encoding="utf-8") as f:
    text = f.read()          # closed automatically

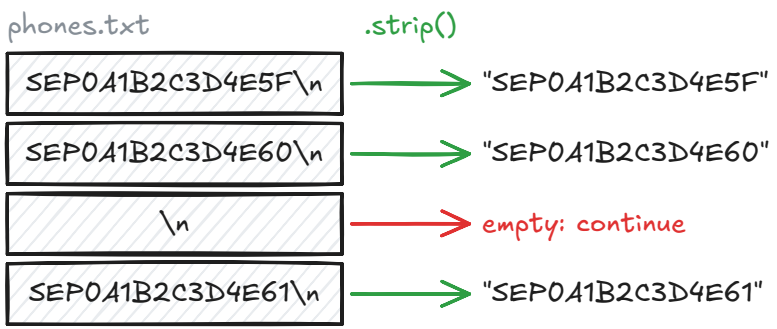
from pathlib import Path
path = Path("data") / "phones.txt"
path.exists()                # True / False
```

`open()` connects your script to a file on disk and returns a **file object**. Always use it with `with`: the file is closed when the block ends, even if an exception happens inside.

Mode	Meaning
`"r"`	read (default), error if the file is missing
`"w"`	write, **erases** existing content
`"a"`	append to the end
`"x"`	create a new file, error if it exists
`"rb"` `"wb"`	binary: bytes instead of str

Pass `encoding="utf-8"` for text, and use `pathlib.Path` to build paths that work on macOS, Linux and Windows.

Reading and writing text files




```python
with open("phones.txt", encoding="utf-8") as f:
    for line in f:              # one line per round
        name = line.strip()     # drop "\n"
        if not name:
            continue            # skip empty lines
        print(name)

with open("report.txt", "w", encoding="utf-8") as f:
    f.write("Phone report\n")
    f.write(f"{name}\n")        # add "\n" yourself
```

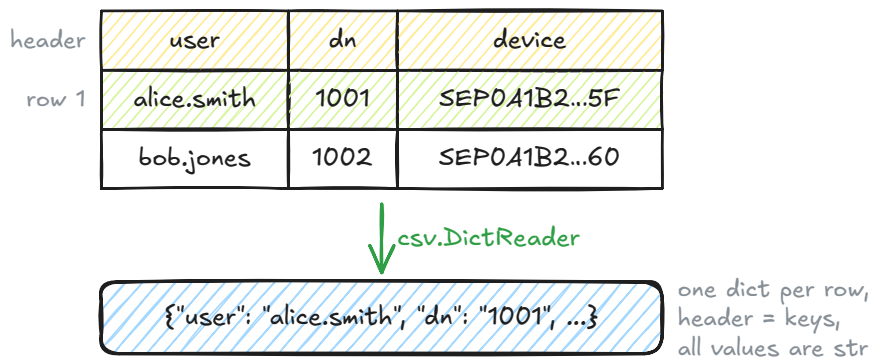
A file object can be looped over **line by line**. Each line still ends with `"\n"`, so `strip()` it first. Reading line by line keeps memory low even for very large files.

Method	What it does
`f.read()`	whole file as one `str`
`for line in f`	one line at a time
`f.readlines()`	`list` of all lines
`f.write(text)`	write a `str`; no newline added
`Path(p).read_text()`	open, read and close in one call

× COMMON MISTAKE

`"w"` empties the file the moment it is opened. Writing the report over your input file destroys it. Use a new file name or `"a"`.

Reading CSV files




```python
import csv

with open("users.csv", newline="",
          encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["user"], row["dn"])
# alice.smith 1001
# bob.jones 1002
```

CSV is the most common input for bulk jobs: every Excel sheet can be saved as one. `csv.DictReader` turns each row into a `dict` and uses the header row as its keys, so you read columns by **name**.

Every value is a `str`. Convert numbers yourself, and open the file with `newline=""` as the `csv` module expects.

> **! WATCH OUT**
> Excel in Turkish or German locale saves with `;` and a hidden BOM. Read such files with `delimiter=";"` and `encoding="utf-8-sig"`, or the first key becomes `"\ufeffuser"`.

Writing CSV files

```python
import csv

results = [
    {"device": "SEP0A1B2C3D4E5F", "status": "added"},
    {"device": "SEP0A1B2C3D4E60", "status": "failed"},
]

with open("result.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["device", "status"])
    writer.writeheader()
    writer.writerows(results)
```

```
device,status
SEP0A1B2C3D4E5F,added
SEP0A1B2C3D4E60,failed
```

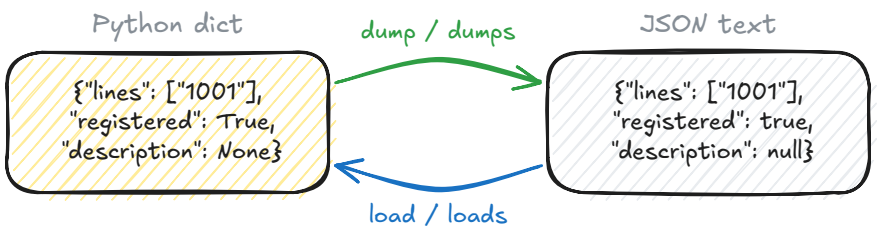
`csv.DictWriter` writes a list of dicts as rows. `fieldnames` sets the column order, `writeheader()` writes the first row.

Tool	Works with
`csv.DictReader`	rows as `dict`, keys from header
`csv.DictWriter`	write `dict` rows
`csv.reader`	rows as `list`, by position
`csv.writer`	write `list` rows

▸ IN THE COLLAB WORLD

Read `users.csv`, provision each row, write `result.csv` with device and status. The same file opens in Excel for the customer.

JSON · load and dump




```python
import json

with open("phone.json", "w", encoding="utf-8") as f:
    json.dump(phone, f, indent=2)   # dict → file

with open("phone.json", encoding="utf-8") as f:
    data = json.load(f)             # file → dict

text = json.dumps(phone)            # dict → str
json.loads(text)["lines"]           # str → dict
```

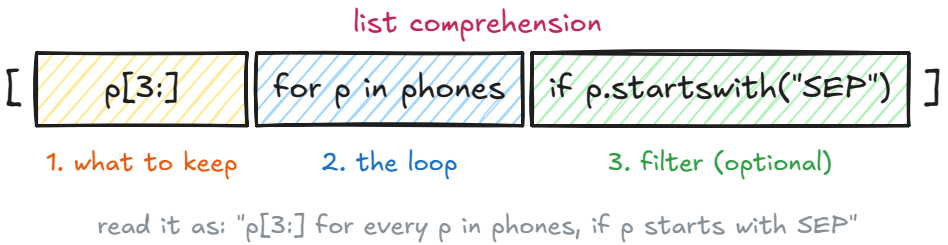
JSON is the text format of REST APIs. Its structure matches Python almost one to one, so the `json` module converts in both directions. The `s` in `dumps` / `loads` means **string**; without it, a file.

Python	JSON
`dict`	object `{}`
`list` (and `tuple`)	array `[]`
`str`	string `""`
`int` `float`	number
`True` `False` `None`	`true` `false` `null`

▸ **IN THE COLLAB WORLD**

Webex APIs send and receive JSON. Saving the current config with `json.dump(..., indent=2)` before a change gives you a readable backup.

List comprehension




```
phones = ["SEP0A1B2C3D4E5F", "CSFALICE",
          "SEP0A1B2C3D4E60"]

desk = []                           # loop version
for p in phones:
    if p.startswith("SEP"):
        desk.append(p[3:])

desk = [p[3:] for p in phones if p.startswith("SEP")]
# ['0A1B2C3D4E5F', '0A1B2C3D4E60']

dns = [str(n) for n in range(1001, 1004)]
```

A **comprehension** builds a new list from another collection in one expression. It replaces the pattern "empty list, loop, `if`, `append`" and reads almost like a sentence.

It has up to three parts, always in this order:

1. **what to keep**: the item itself or a transformed value,
2. **the loop**: `for item in collection`,
3. **a filter** (optional): `if condition`.

The original list is not changed: you always get a **new** list.

> ▸ **IN THE COLLAB WORLD**
> One line turns a phone list into the MAC list a Webex device onboarding sheet needs, or keeps only the desk phones of a site.

Dict and set comprehensions

```
rows = [{"user": "alice.smith", "dn": "1001"},
        {"user": "bob.jones",   "dn": "1002"},
        {"user": "carol.white", "dn": "1003"}]

dn_of = {r["user"]: r["dn"] for r in rows}    # dict
dn_of["bob.jones"]                            # '1002'

dns = ["1001", "1002", "2000"]
{dn[:2] for dn in dns}                        # set
# {'10', '20'}  DN ranges in use

statuses = ["Registered", "UnRegistered", "Registered"]
["up" if s == "Registered" else "down" for s in statuses]
# ['up', 'down', 'up']

sum(1 for s in statuses if s != "Registered")    # 1 down
all(p.startswith("SEP") for p in desk)           # True
```

The same idea works for every collection. The brackets decide what you get back:

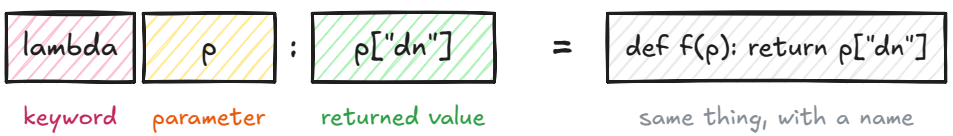
Form	Result
`[x for …]`	`list`
`{k: v for …}`	`dict`
`{x for …}`	`set`, duplicates removed
`(x for …)`	generator: one item at a time

A generator inside `sum()`, `any()` or `all()` needs no extra brackets.

▸ **IN THE COLLAB WORLD**

`{r["user"]: r["dn"] for r in rows}` turns a CSV into a lookup table: find any user's DN instantly, without a loop.

lambda and key functions




```
phones = [{"name": "SEP0A1B2C3D4E60", "dn": "1002"},
          {"name": "SEP0A1B2C3D4E5F", "dn": "1001"}]
sorted(phones, key=lambda p: p["dn"])  # by DN

cdrs = [{"caller": "1001", "duration": 125},
        {"caller": "1002", "duration": 410}]
max(cdrs, key=lambda c: c["duration"])
# {'caller': '1002', 'duration': 410}
```

A `lambda` is a tiny function without a name: one parameter list, one expression, returned automatically. It is used where a function is needed for a moment only.

Its most common job is the `key=` argument. `sorted`, `min` and `max` call the key function on every item and compare the results instead of the items themselves.

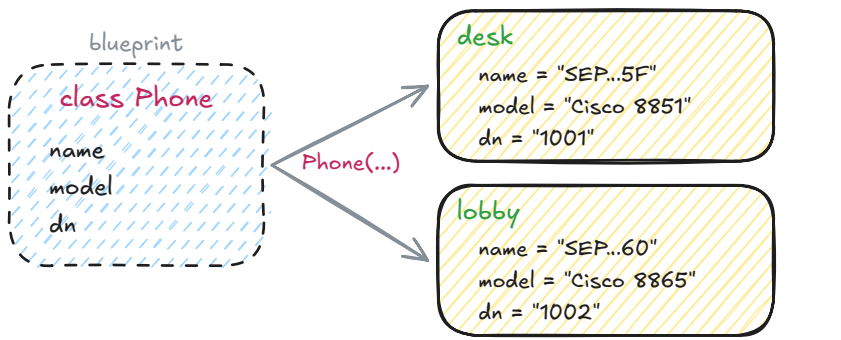
× COMMON MISTAKE

If a comprehension or lambda no longer fits on one readable line, write a normal loop or `def`. Short and clear beats clever.

▸ IN THE COLLAB WORLD

Sort phones by DN for a report, or find the longest call in a CDR export with `max(cdrs, key=…)`.

Classes and objects




```python
class Phone:
    def __init__(self, name, model, dn):
        self.name = name        # attributes
        self.model = model
        self.dn = dn

desk = Phone("SEP0A1B2C3D4E5F", "Cisco 8851", "1001")
lobby = Phone("SEP0A1B2C3D4E60", "Cisco 8865", "1002")

desk.dn          # '1001'
lobby.model      # 'Cisco 8865'
```

A **class** is a blueprint for your own type. It says which data (**attributes**) and which behaviour (**methods**) every object of that type has. Each **object** (instance) built from it has its own values.

Part	Meaning
`class Phone:`	define a new type (CamelCase name)
`__init__`	runs when an object is created
`self.name = …`	store an attribute on the object
`Phone(...)`	create a new object
`desk.dn`	read an attribute with a dot

▸ **IN THE COLLAB WORLD**

A `dict` holds a phone's data. A class holds the data **and** the rules that belong to it, so every script treats a phone the same way.

Methods and self




```python
class Phone:
    VIDEO_MODELS = ("Cisco 8845", "Cisco 8865")  # shared

    def __init__(self, name, model, dn):
        self.name, self.model, self.dn = name, model, dn
        self.status = "Unknown"

    def mac(self):
        return self.name[3:]

    def is_video(self):
        return self.model in self.VIDEO_MODELS

desk = Phone("SEP0A1B2C3D4E5F", "Cisco 8851", "1001")
desk.mac()                  # '0A1B2C3D4E5F'
desk.is_video()             # False
desk.status = "Registered"  # attributes can change
```

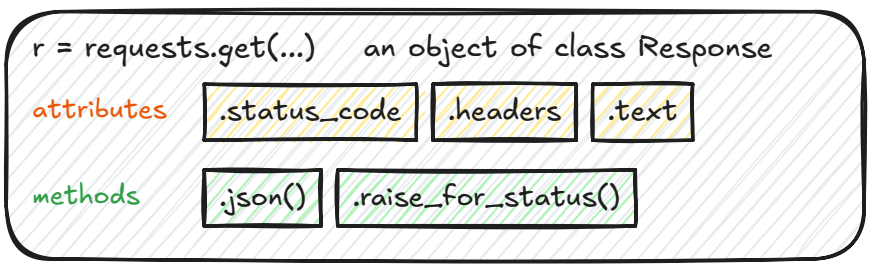
A **method** is a function defined inside a class. Its first parameter, `self`, is the object the method was called on. Python fills it in: `desk.mac()` runs as `Phone.mac(desk)`.

Through `self` a method reads and changes the object's own attributes. A **class attribute** like `VIDEO_MODELS` is shared by all objects.

Names with double underscores are special: `__init__` builds the object, `__repr__` decides how `print(desk)` looks.

> **× COMMON MISTAKE**
> Forgetting `self` in `def mac():` raises a `TypeError`: "takes 0 positional arguments but 1 was given".

Classes in the libraries you will use




```python
class AxlClient:
    def __init__(self, host, user, password):
        self.url = f"https://{host}:8443/axl/"
        self.auth = (user, password)

    def get_phone(self, name):
        ...         # send the AXL request here

axl = AxlClient("cucm-pub.clauto.lab",
                "axl_svc", password)
axl.get_phone("SEP0A1B2C3D4E5F")
```

Most of the time you **use** classes rather than write them. Every library hands you objects: you read their attributes and call their methods with a dot.

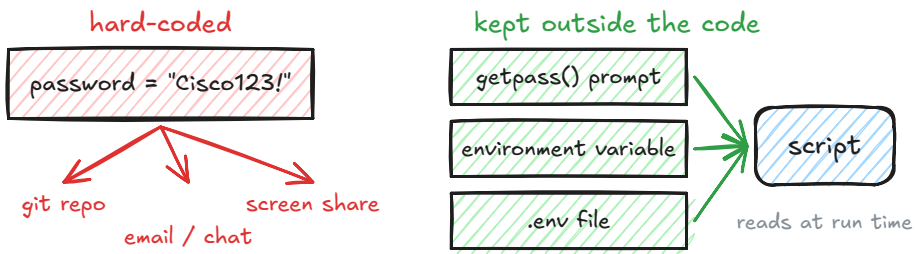
Object	From
`Response`	`requests.get()` : `.status_code` , `.json()`
`Session`	`requests.Session()` : reuses login and connection
`Client`	`zeep.Client(wsdl)` : `.service.getPhone()`

Writing a small class of your own pays off when several functions share the same settings, like host and credentials.

▸ **IN THE COLLAB WORLD**

An `AxlClient` stores the CUCM host and login once. Every script then just calls `axl.get_phone(...)` or `axl.add_line(...)`.

Keep secrets out of your code




```
password = "Cisco123!"          # never do this

from getpass import getpass
password = getpass("AXL password: ")   # not echoed

import os
user = os.environ["AXL_USER"]   # KeyError if not set
password = os.environ.get("AXL_PASSWORD")  # or None
```

```
export AXL_USER=axl_svc         # in the terminal
```

A password written in a script travels with the script: into git, into an email to a colleague, onto a shared screen. Keep secrets **outside** the code and read them when the script runs.

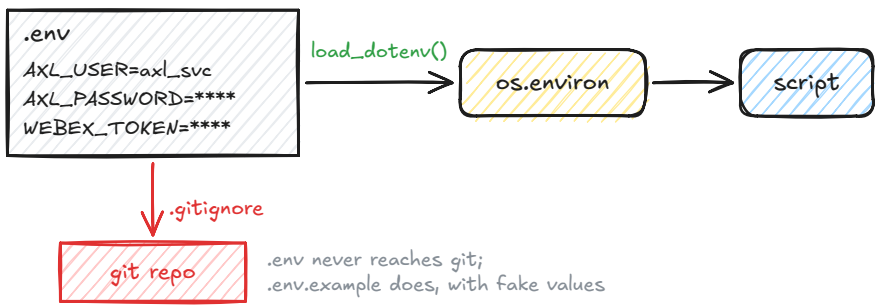
Source	Good for
`getpass()`	interactive runs: asks, shows nothing
`os.environ`	values set in the shell or by a scheduler
`.env` file	per-project settings on your laptop

`os.environ` behaves like a `dict` of strings: `[...]` fails on a missing name, `.get()` returns `None`.

▸ **IN THE COLLAB WORLD**

Use a dedicated AXL application user (`axl_svc`) with only the **Standard AXL API Access** and **Standard CCM Admin Users** roles, never your own admin account.

.env files and a safe setup




```
# .env  (in .gitignore)
CUCM_HOST=cucm-pub.clauto.lab
AXL_USER=axl_svc
AXL_PASSWORD=change-me
```

```python
# pip install python-dotenv
import os
from dotenv import load_dotenv

load_dotenv()                       # .env → os.environ
missing = [k for k in ("AXL_USER", "AXL_PASSWORD")
           if k not in os.environ]
if missing:
    raise SystemExit(f"missing settings: {missing}")
```

`python-dotenv` reads a `.env` file and puts every line into `os.environ`. The code stays the same on your laptop, a jump host or a scheduler; only the `.env` changes.

Checking all required settings **at the start** gives one clear message instead of a failure halfway through a bulk job.

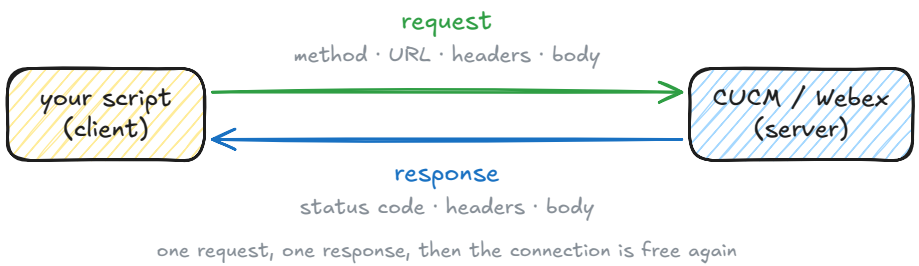
Do	Why
add `.env` to `.gitignore`	secrets never reach the repo
commit `.env.example`	others see which names are needed
one AXL user per tool	easy to audit and to revoke
never `print` a secret	logs and screens are shared

2 APIs and Libraries

Every Cisco collaboration product can be driven through an API: CUCM through AXL and UDS, Unity Connection through CUPI, Webex through its REST API. This chapter covers the web basics behind all of them, the data formats they speak, and the two Python libraries that do the heavy lifting: `requests` for REST and `zeep` for SOAP.

2.1 HTTP Basics
2.2 JSON and XML
2.3 requests
2.4 zeep and AXL
2.5 RisPort70
2.6 Netmiko
2.7 CMS REST API
2.8 Remote Phone Control
2.9 Wrap-up

What is an API?




```
GET /v1/people?email=alice.smith@clauto.lab HTTP/1.1
Host: webexapis.com
Authorization: Bearer <token>
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json

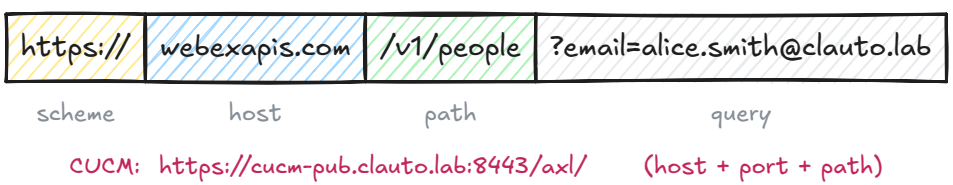
{"items": [{"displayName": "Alice Smith", ...}]}
```

An **API** (application programming interface) is a door into a system made for programs instead of people. Where an admin clicks through the GUI, a script sends the same request directly.

Almost every modern API runs over **HTTP(S)**: the client sends a **request**, the server answers with a **response**. Both are plain text with the same structure: a first line, headers, an optional body.

API	Style
CUCM AXL, RISPort	SOAP: XML over HTTPS
CUCM UDS, Unity CUPI	REST: XML or JSON
Webex	REST: JSON

Methods and URLs




```
GET    /v1/people?email=…    find a user
POST   /v1/messages          send a message
PUT    /v1/people/{id}       replace a user record
DELETE /v1/people/{id}       remove a user
```

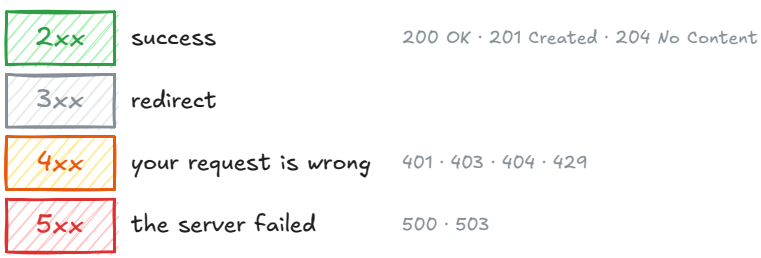
The **method** says what to do, the **URL** says with what. Together they form the first line of every request.

Method	Meaning
`GET`	read data, changes nothing
`POST`	create something new, or run an action
`PUT`	replace an existing record completely
`PATCH`	change part of a record
`DELETE`	remove a record

The **query string** after `?` filters or pages the result: `?email=…`, `?max=100`.

> ▸ **IN THE COLLAB WORLD**
> SOAP APIs like AXL use `POST` for everything, even for reads. The operation (`getPhone`, `addLine`) is written inside the XML body.

Status codes, headers and auth




```
Content-Type: application/json    # what I send
Accept: application/json          # what I want back
Authorization: Basic YXhsX3N2YzpwYXNzd29yZA==
Authorization: Bearer <access token>
```

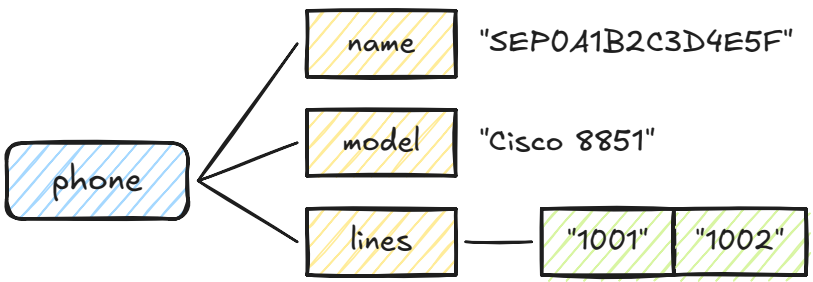
The **status code** in the first response line tells you at once if the request worked. **Headers** carry extra information for both sides, most importantly the format and the login.

Code	What it usually means for you
`401`	wrong user / password, or token expired
`403`	login OK, but the role is missing
`404`	wrong URL or the record does not exist
`429`	too many requests: wait `Retry-After` seconds
`500` `503`	server error or service down

▸ **IN THE COLLAB WORLD**

AXL uses **Basic** auth (base64 of `user:password`). Webex uses a **Bearer** token. Lab CUCMs have self-signed certificates, so certificate checks need extra care.

Same data, two formats




```
{"phone": {"name": "SEP0A1B2C3D4E5F",
           "model": "Cisco 8851",
           "lines": ["1001", "1002"]}}
```

```
<phone>
  <name>SEP0A1B2C3D4E5F</name>
  <model>Cisco 8851</model>
  <lines><line>1001</line><line>1002</line></lines>
</phone>
```

APIs send structured data as text. Both formats describe the same **tree**: a phone with a name, a model and a list of lines.

JSON	XML
object `{"k": v}`	element `<k>v</k>`
array `[a, b]`	the same element repeated
`true` `null` numbers	everything is text
compact, maps to `dict` / `list`	verbose, has attributes and namespaces

JSON maps directly to Python with `json.loads()` (see 1.8.2). XML needs a parser that walks the tree.

> ▸ **IN THE COLLAB WORLD**
> Webex and most new APIs use JSON. AXL, RIS and many CUCM and Unity APIs answer in XML, so you need both.

Reading XML with ElementTree



element = start tag + content + end tag; elements nest inside elements

```
import xml.etree.ElementTree as ET

root = ET.fromstring(xml_text)      # str → tree
root.get("totalCount")              # '2' (attribute)

for user in root.findall("user"):
    print(user.findtext("userName"),
          user.findtext("phoneNumber"))
# alice.smith 1001
# bob.jones 1002

[e.text for e in root.iter("phoneNumber")]
```

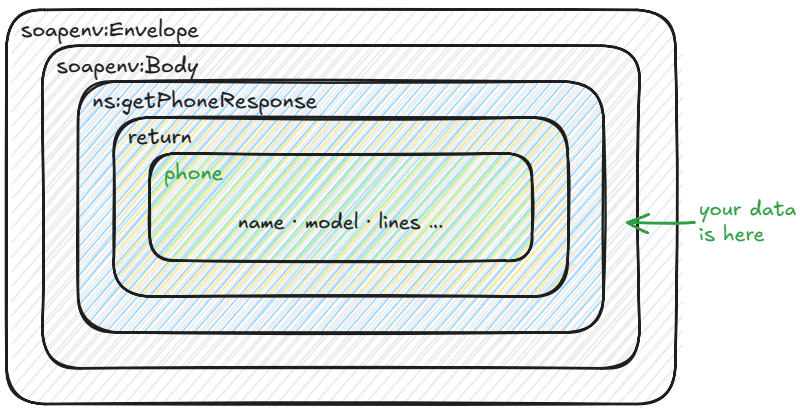
`ElementTree` (standard library) turns XML text into a tree of **elements**. Each element has a `tag`, optional **attributes** and a **text**. You search the tree with simple paths like `"user/userName"`.

Method	What it does
`ET.fromstring(s)`	parse text, returns the root element
`el.find(path)`	first matching child, or `None`
`el.findall(path)`	`list` of all matches
`el.findtext(path)`	text of the first match
`el.get(name)`	an attribute value
`el.iter(tag)`	every element with that tag, any depth

▸ **IN THE COLLAB WORLD**

CUCM UDS returns users as XML like this: one `<user>` element per user, with `<userName>` and `<phoneNumber>` inside.

Namespaces and the SOAP envelope




```
SOAP = "http://schemas.xmlsoap.org/soap/envelope/"

root = ET.fromstring(axl_reply)
root.tag                          # '{' + SOAP + '}Envelope'
root.find(".//phone/name").text # 'SEP0A1B2C3D4E5F'

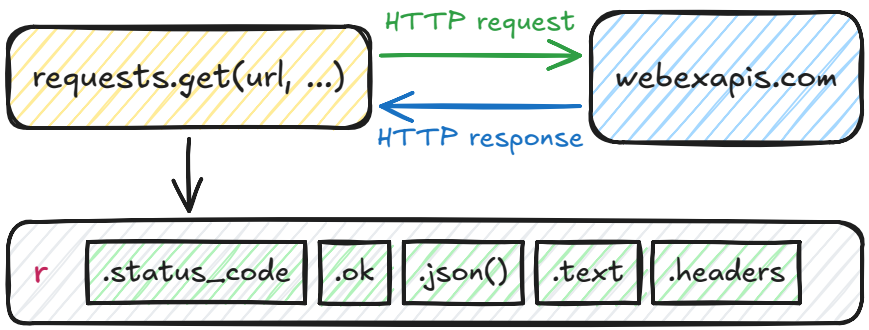
# prefixed tags need the namespace map
root.find("soapenv:Body", {"soapenv": SOAP})
```

A **namespace** is a URL that makes tag names unique: `soapenv:Body` really means `{http://…/envelope/}Body`. The short prefix (`soapenv:`, `ns:`) is only an alias declared with `xmlns:`.

A SOAP reply wraps your data in layers: `Envelope`, `Body`, the operation response, then `return`. The `.//` path searches at any depth and skips the wrapping.

> ▸ **IN THE COLLAB WORLD**
> AXL replies are SOAP envelopes, and AXL requests must be built the same way. Writing them by hand is slow and error-prone. `zeep` builds and unpacks them for you: that is the next section.

Your first request




```python
import os
import requests          # pip install requests

token = os.environ["WEBEX_TOKEN"]
r = requests.get(
    "https://webexapis.com/v1/people/me",
    headers={"Authorization": f"Bearer {token}"},
    timeout=10,
)
r.status_code            # 200
r.json()["displayName"]  # 'Alice Smith'
r.raise_for_status()     # error if 4xx / 5xx
```

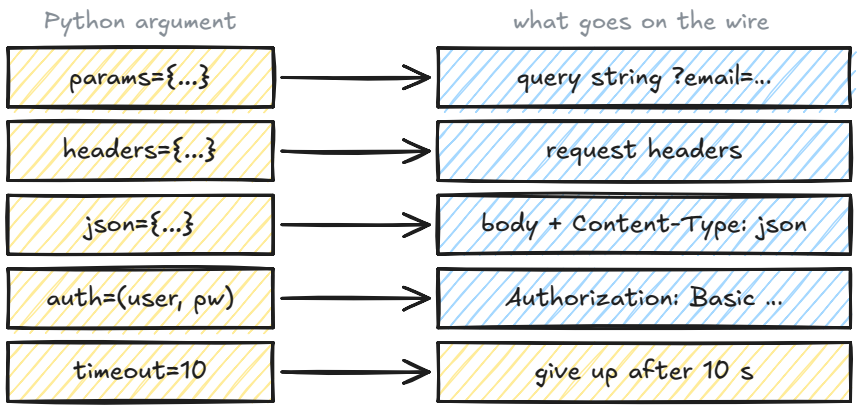
`requests` turns an HTTP call into one function call. There is one function per method: `get`, `post`, `put`, `patch`, `delete`. Each returns a `Response` object.

Response	Gives you
`r.status_code` `r.ok`	the code, `True` for 2xx
`r.json()`	body parsed into `dict` / `list`
`r.text` `r.content`	body as `str` / `bytes`
`r.headers`	response headers (a dict)
`r.raise_for_status()`	raise `HTTPError` on 4xx / 5xx

▸ **IN THE COLLAB WORLD**

`GET /v1/people/me` is the “hello world” of the Webex API: if it returns your name, the token works.

Parameters, headers and JSON bodies




```
url = "https://webexapis.com/v1"
headers = {"Authorization": f"Bearer {token}"}

query = {"email": "alice.smith@clauto.lab"}
r = requests.get(f"{url}/people", headers=headers,
                 params=query, timeout=10)
r.json()["items"][0]["displayName"]

msg = {"toPersonEmail": "bob.jones@clauto.lab",
       "text": "SEP0A1B2C3D4E5F is down"}
r = requests.post(f"{url}/messages", headers=headers,
                  json=msg, timeout=10)
```

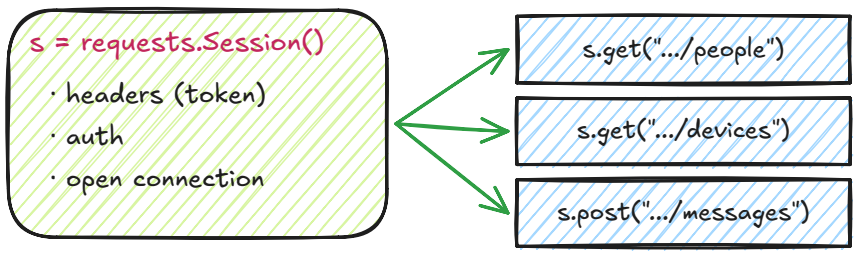
Keyword arguments map one to one onto the parts of the HTTP request. `requests` does the encoding: it builds the query string, turns the `dict` into JSON and sets `Content-Type`.

- `params=` for filters in the URL. Never glue them in by hand.
- `json=` for a JSON body (`POST`, `PUT`, `PATCH`).
- `headers=` for tokens and formats.

× COMMON MISTAKE

Without `timeout=` a request can wait **forever** if the server does not answer. Always set one, around 10 to 30 seconds.

Auth, sessions and certificates



set once, reused by every call: less code, faster

```python
r = requests.get(                       # Basic auth
    "https://cuc-pub.clauto.lab/vmrest/users",
    auth=("axl_svc", password),
    headers={"Accept": "application/json"},
    params={"query": "(alias is alice.smith)"},
    verify="certs/clauto-ca.pem",       # trust lab CA
    timeout=10)

s = requests.Session()
s.headers.update({"Authorization": f"Bearer {token}"})
s.get(f"{url}/people/me", timeout=10)   # reused
```

`auth=(user, password)` sends Basic auth, used by CUCM UDS and Unity CUPI. A `Session` keeps headers, auth and the TCP connection between calls: less code, and much faster in loops.

`verify=` controls the TLS certificate check. Point it to your CA certificate file, so the check keeps protecting you.

> **! WATCH OUT**
> `verify=False` switches the check off and prints an `InsecureRequestWarning`. Acceptable for a first lab test, never for production scripts.

Errors and timeouts

```python
import requests

try:
    r = s.get(f"{url}/people/me", timeout=10)
    r.raise_for_status()
except requests.exceptions.Timeout:
    print("no answer in 10 s")
except requests.exceptions.ConnectionError:
    print("host unreachable: DNS, VPN, firewall?")
except requests.exceptions.HTTPError as e:
    print("API error:", e)
    # 401 Client Error: Unauthorized for url: …
else:
    me = r.json()
```

A request can fail in three different places, and `requests` raises a different exception for each. All of them live in `requests.exceptions`.

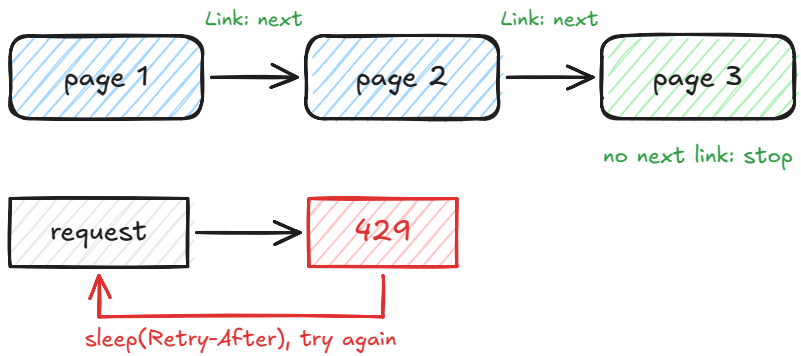
Case	Handle with
no answer in time	`timeout=` + `except Timeout`
host unreachable	`except ConnectionError`
answer is 4xx / 5xx	`raise_for_status()` + `except HTTPError`

A 4xx or 5xx answer is **not** an exception by itself: `r.status_code` is simply `401`. Call `raise_for_status()` to turn it into one.

▸ **IN THE COLLAB WORLD**

In a bulk job, catch these inside the loop (see 1.7) and write the device and the error to `failed.csv`.

Rate limits and paging




```python
import time

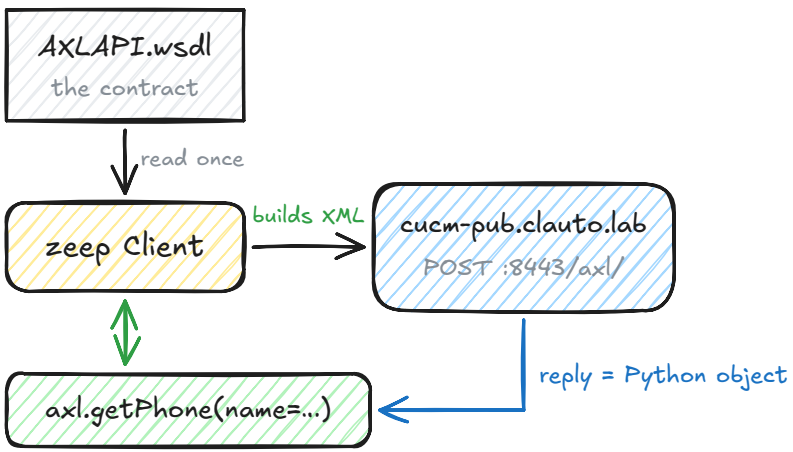
people, next_url = [], f"{url}/people?max=100"
while next_url:
    r = s.get(next_url, timeout=10)
    if r.status_code == 429:            # slow down
        wait = int(r.headers.get("Retry-After", 5))
        time.sleep(wait)
        continue                        # same page
    r.raise_for_status()
    people += r.json()["items"]
    next_url = r.links.get("next", {}).get("url")
```

APIs protect themselves. **Rate limits** cap how many requests you may send: above the limit you get `429 Too Many Requests`, and the `Retry-After` header says how long to wait.

Paging splits long lists: each response carries one page, and a `Link` header with `rel="next"` points to the next one. Follow it until there is no next link.

> ▸ **IN THE COLLAB WORLD**
> Listing all users of a Webex org is a paging loop. `r.links` parses the `Link` header for you: `r.links["next"]["url"]`.

SOAP, WSDL and zeep




```
schema/
└── 15.0/                    one folder per CUCM version
    ├── AXLAPI.wsdl
    ├── AXLSoap.xsd
    └── AXLEnums.xsd
```

```
pip install zeep
```

A **WSDL** file is the contract of a SOAP API: every operation, every field and its type. `zeep` reads it once and turns each operation into a Python method, so `axl.getPhone(name=…)` builds the SOAP envelope, sends it and turns the reply into an object.

Prerequisite	Where
Cisco AXL Web Service active	Serviceability, on the Publisher
AXL application user	roles: Standard AXL API Access, Standard CCM Admin Users
WSDL for **your** version	CM Admin: Application → Plugins → AXL Toolkit
Tomcat CA certificate	OS Admin: Certificate Management

! WATCH OUT

The WSDL must match the cluster version. An older schema “mostly works” and silently misses newer fields. Re-test after every upgrade.

Connecting to AXL

```
from requests import Session
from zeep import Client, Settings
from zeep.cache import SqliteCache
from zeep.transports import Transport

BINDING = "{http://www.cisco.com/AXLAPIService/}AXLAPIBinding"
URL = "https://cucm-pub.clauto.lab:8443/axl/"

session = Session()
session.auth = (user, password)       # from os.environ (1.11)
session.verify = "certs/clauto-ca.pem"
transport = Transport(session=session, timeout=30,
                      cache=SqliteCache())
settings = Settings(strict=False, xml_huge_tree=True)
client = Client("schema/15.0/AXLAPI.wsdl",
                transport=transport, settings=settings)
axl = client.create_service(BINDING, URL)

axl.getCCMVersion()["return"]["componentVersion"]["version"]
# '15.0.1.11900-23'
```

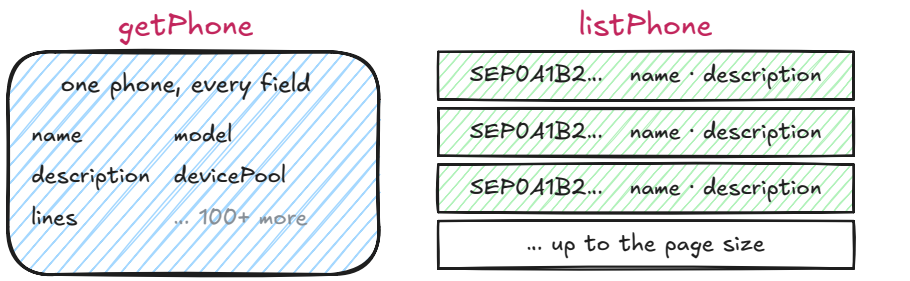
The connection is built in layers. A `requests` `Session` carries the login and the certificate check. A `Transport` adds caching and a timeout. The `Client` reads the WSDL, and `create_service` points it at your Publisher.

`SqliteCache` keeps the parsed WSDL on disk: the AXL schema is large, and parsing it on every run is slow.

Run `getCCMVersion` first. If it returns a version, four things are proven at once: network, credentials, roles and certificate.

> ▸ **IN THE COLLAB WORLD**
> Wrap these lines in one `get_axl()` function or the `AxlClient` class from 1.10. Every script then starts with one line.

Reading one phone · getPhone




```python
from zeep.helpers import serialize_object

resp = axl.getPhone(name="SEP0A1B2C3D4E5F")
phone = resp["return"]["phone"]

phone.name                # 'SEP0A1B2C3D4E5F'
phone.devicePoolName      # {'_value_1': 'DP_HQ',
                          #  'uuid': '{…}'}
def axl_text(value):      # unwrap safely
    return getattr(value, "_value_1", value)

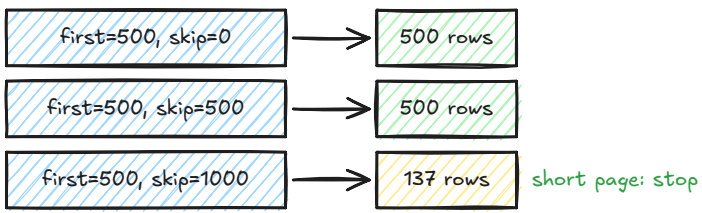
axl_text(phone.devicePoolName)   # 'DP_HQ'
serialize_object(phone, dict)    # plain dict
```

`get` operations return **one** object with every field. The data is always inside `resp["return"]`, under the object name (`phone`, `line`, `user`). zeep objects work with dots and with brackets.

Fields that point to other objects (device pool, CSS, location) come back as `{_value_1: name, uuid: …}`. Read them through one helper like `axl_text()`, never with `._value_1` all over the code: the shape can change between AXL versions.

> **× COMMON MISTAKE**
> `getPhone` for a name that does not exist raises a `Fault`, not `None`. Wrap it in `try` / `except Fault` (next slides).

Reading many phones · listPhone




```python
def list_all(method, criteria, tags, page=500):
    skip = 0
    while True:
        resp = method(searchCriteria=criteria,
                      returnedTags=tags,
                      first=page, skip=skip)
        rows = resp["return"]
        batch = rows["phone"] if rows else []
        yield from batch
        if len(batch) < page:
            return
        skip += page

tags = {"name": "", "description": ""}
for p in list_all(axl.listPhone, {"name": "SEP%"},
                  tags):
    print(p.name, p.description)
```

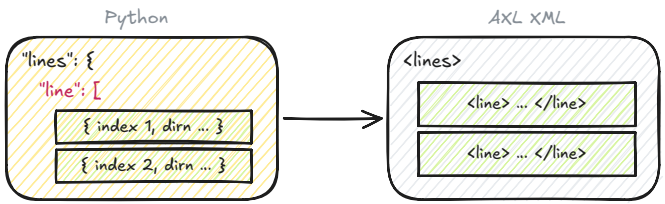
`list` operations return **many** objects, but only the fields you name in `returnedTags`. Three rules keep the Publisher healthy:

Rule	Why
always set `returnedTags`	every field × 5000 phones = overload
narrow `searchCriteria`	`%` alone matches the whole cluster
page with `first` / `skip`	stop when a page comes back short

`yield from` makes `list_all` a **generator**: it hands out one phone at a time, page after page, without holding them all in memory.

> ▸ **IN THE COLLAB WORLD**
> `list_all` works for `listLine`, `listUser` and the other `list` operations too: only the result key changes.

Adding objects · nested data



plural wrapper, singular items: lines/line, members/member, speeddials/speeddial

```
phone = {
    "name": "CSFALICE",
    "product": "Cisco Unified Client Services Framework",
    "class": "Phone", "protocol": "SIP", "protocolSide": "User",
    "commonPhoneConfigName": "Standard Common Phone Profile",
    "locationName": "Hub_None", "devicePoolName": "DP_HQ",
    "phoneTemplateName": "Standard Client Services Framework",
    "ownerUserName": "alice.smith",
    "lines": {"line": [
        {"index": "1", "label": "1001",
         "dirn": {"pattern": "1001",
                  "routePartitionName": "PT_Internal"}},
    ]},
}
axl.addPhone(phone=phone)      # returns the new uuid
```

`add` operations take **one** argument named after the object (`phone=`, `user=`, `css=`, `sipTrunk=`) whose value is a nested `dict` that mirrors the AXL XML. zeep turns it into the request element by element.

Every repeating element follows the same pattern: a **plural** wrapper key holding a **list** under the **singular** key.

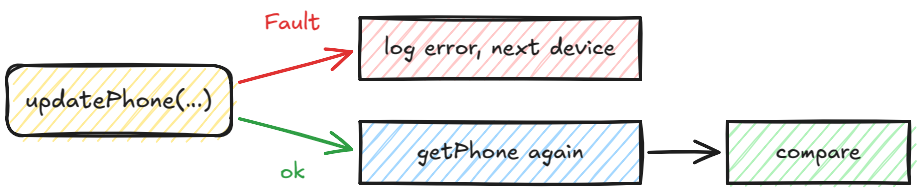
× COMMON MISTAKE

`updatePhone(name=…, lines={"line": […]})` **replaces** the whole line list. Send every line the phone should keep, not only the new one.

▸ IN THE COLLAB WORLD

The same shape adds partitions in a loop (`addRoutePartition`), CSS members (`members/member`) or trunk destinations.

Writing data and handling faults



a successful reply only means CUCM accepted the request

```python
import time
from zeep.exceptions import Fault

for name, desc in updates.items():
    try:
        axl.updatePhone(name=name, description=desc)
    except Fault as e:
        failed.append((name, e.message))
        continue
    now = axl.getPhone(name=name)["return"]["phone"]
    if now.description != desc:
        failed.append((name, "not applied"))
    time.sleep(0.2)          # spare the Publisher
```

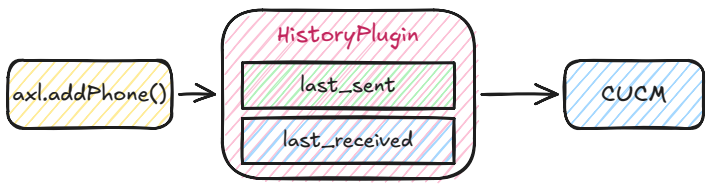
`add`, `update` and `remove` operations change the database. Treat every write the same way: catch the `Fault`, re-read to **verify**, and pause between writes.

Fault text (varies by release)	Usual cause
`Item not valid … not found`	typo in a name, pool or CSS
`duplicate value`	the object already exists
`User not authorized`	AXL role missing
HTTP `503`	write throttling: wait, then retry

! WATCH OUT

Send writes to the **Publisher** only. AXL is not real time and throttles bursts. It also does not know registration state: that lives in RisPort70.

Seeing the raw XML · HistoryPlugin



keeps a copy of the raw XML in both directions

```
from lxml import etree
from zeep.plugins import HistoryPlugin

history = HistoryPlugin()
client = Client(wsdl, transport=transport, plugins=[history])

def show_history():
    for item in (history.last_sent, history.last_received):
        print(etree.tostring(item["envelope"], encoding="unicode",
                             pretty_print=True))

try:
    axl.addPhone(phone=phone)
except Fault as f:
    print(f.message)
    show_history()          # what exactly was sent?
```

When AXL answers with a `Fault`, the message alone is often not enough. `HistoryPlugin` keeps the last request and the last response as XML, so you can see exactly what zeep sent and what CUCM returned.

Compare the sent XML with the AXL schema reference or with a request that works in SoapUI: missing fields, wrong nesting or a typo in an element name show up immediately.

> ★ **TIP**
> Call `show_history()` inside every `except Fault` while a script is new. Remove it, or log it to a file, once the script is stable.

SQL queries · executeSQLQuery

```
sql = """
select d.name, n.dnorpattern
from device d
join devicenumplanmap m on m.fkdevice = d.pkid
join numplan n on n.pkid = m.fknumplan
where d.name like 'SEP%'
"""
resp = axl.executeSQLQuery(sql=sql)
rows = resp["return"]["row"] if resp["return"] else []

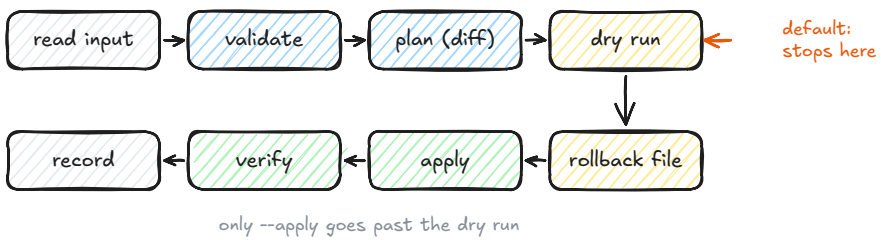
for row in rows:                # row = list of XML elements
    print({el.tag: el.text for el in row})
# {'name': 'SEP0A1B2C3D4E5F', 'dnorpattern': '1001'}
# {'name': 'SEP0A1B2C3D4E60', 'dnorpattern': '1002'}
```

Thick AXL runs a read-only SQL query on the CUCM database and returns one `row` per result line. Each row is a list of XML elements: `el.tag` is the column name, `el.text` the value.

One query can join tables that would need many `list` calls, which makes it ideal for cluster-wide reports.

> **! WATCH OUT**
> Prefer normal AXL methods when one exists: table and column names change between releases (check the **Data Dictionary** for yours). Never use `executeSQLUpdate`: it bypasses CUCM's business logic.

A safe AXL workflow




```python
import argparse

p = argparse.ArgumentParser()
p.add_argument("--input-file", required=True)
p.add_argument("--apply", action="store_true")
args = p.parse_args()

changes = plan(axl, read_input(args.input_file))
print_plan(changes)
if not args.apply:
    raise SystemExit("DRY RUN: use --apply to write")
write_rollback(changes)          # old values first
apply_and_verify(axl, changes)
```

A script that only writes is dangerous. A script that checks, shows the plan, writes, re-checks and records is a tool you can trust in a change window.

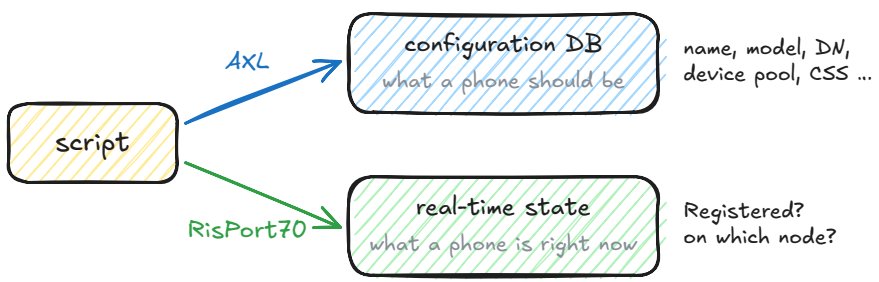
Use	When
BAT	uniform change, fits a BAT template
Python + AXL	each object needs a decision, a check, or data from another system

Dry run is the default. Writing needs an explicit `--apply`, and the old values are saved before the first write.

▸ **IN THE COLLAB WORLD**

Working examples: `github.com/ilkermansur/collab_automation` (AXL, RIS, CMS, Expressway, Netmiko). Read more: “Automating Cisco Voice Administration (CUCM) with Python”, Cisco Community, 2026.

Real-time status with RisPort70




```
HOST = "https://cucm-pub.clauto.lab:8443"
RIS = HOST + "/realtimeservice2/services/RISService70"
NS = "{http://schemas.cisco.com/ast/soap}"

client = Client(RIS + "?wsdl", transport=transport)
ris = client.create_service(NS + "RisBinding", RIS)
```

AXL only knows the **configuration**: a phone exists, has DN 1001, sits in `DP_HQ`. Whether it is registered right now lives in the **Real-time Information Service** (RIS). RisPort70 is its SOAP API.

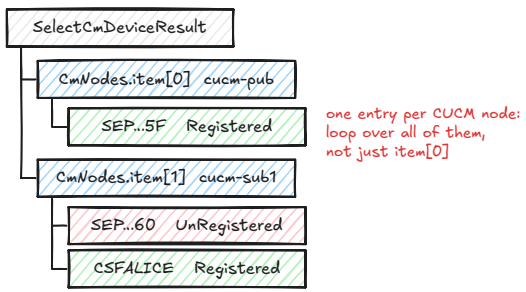
It works with the same tools as AXL: a `requests` `Session`, a zeep `Transport` and `create_service`. The WSDL is served by CUCM itself, so there is no toolkit to download.

AXL	RisPort70
`/axl/`	`/realtimeservice2/services/RISService70`
WSDL from the AXL Toolkit	WSDL from `…/RISService70?wsdl`
config: what it **should** be	state: what it **is** now

▸ **IN THE COLLAB WORLD**

The application user needs the **Standard CCM Admin Users** and **Standard RealtimeAndTraceCollection** roles.

Asking for device status · selectCmDevice




```python
def ris_status(names):
    criteria = {
        "MaxReturnedDevices": 1000, "DeviceClass": "Phone",
        "Model": 255, "Status": "Any",   # 255 = any model
        "SelectBy": "Name",
        "SelectItems": {"item": [{"Item": n} for n in names]},
    }
    resp = ris.selectCmDevice(StateInfo="",
                              CmSelectionCriteria=criteria)
    status = {}
    for node in resp["SelectCmDeviceResult"]["CmNodes"]["item"]:
        found = node["CmDevices"]    # None if the node has none
        for dev in (found["item"] if found else []):
            status[dev["Name"]] = (dev["Status"], node["Name"])
    return status
```

`selectCmDevice` takes **selection criteria**: which device class, which status, and a list of names (wildcards like `SEP*` work too).

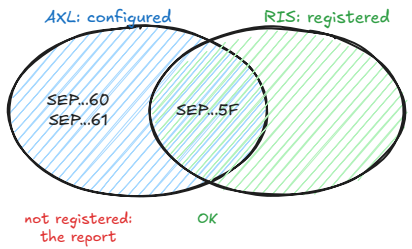
The answer is grouped **per CUCM node**. A phone appears under the node it is registered to, so loop over every node.

Status	Meaning
`Registered`	online on that node
`UnRegistered`	was registered, now gone
`Rejected`	CUCM refused the registration

× COMMON MISTAKE

Reading only `CmNodes["item"][0]` checks the Publisher alone. Phones registered to a Subscriber are silently missed.

Limits and an unregistered-phone report




```
def chunks(items, size=1000):        # max per request
    for i in range(0, len(items), size):
        yield items[i:i + size]

phones = list_all(axl.listPhone, {"name": "SEP%"},
                  {"name": ""})              # AXL
names = [p.name for p in phones]
status = {}
for part in chunks(names):
    status.update(ris_status(part))          # RIS
    time.sleep(5)                            # limit

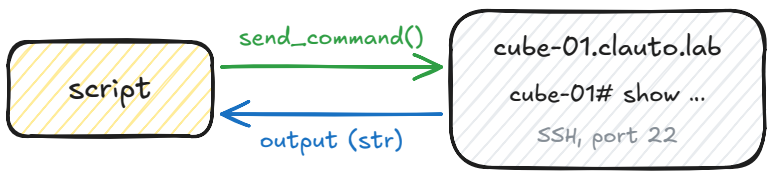
for name in names:
    state, node = status.get(name, ("not seen", "-"))
    if state != "Registered":
        print(name, state, node)
```

RIS protects the cluster with hard limits: at most **1000 devices per request**, and by default about **15 requests per minute**. Larger inventories are sent in chunks with a pause in between.

A phone that has not tried to register since the last CUCM restart is not in RIS at all. Only joining with the AXL inventory finds it: configured but “not seen”.

> ▸ **IN THE COLLAB WORLD**
> Run this report every morning: the AXL list says what should be online, RIS says what is. The difference is the list for the site team.

CLI automation over SSH



Netmiko types the command, waits for the prompt, returns the text

```python
import os
from netmiko import ConnectHandler   # pip install netmiko

cube = {
    "device_type": "cisco_xe",
    "host": "cube-01.clauto.lab",
    "username": os.environ["CUBE_USER"],
    "password": os.environ["CUBE_PASSWORD"],
}

with ConnectHandler(**cube) as conn:     # SSH login
    print(conn.send_command("show version | include uptime"))
# cube-01 uptime is 12 weeks, 3 days, 4 hours
```

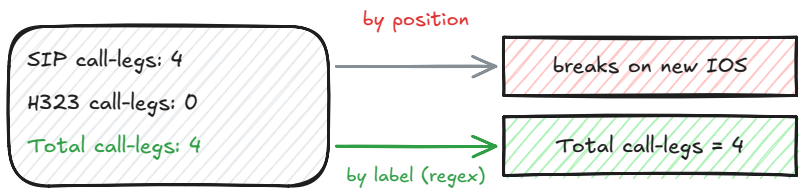
Not every box has an API. CUBE and IOS voice gateways are configured through the **CLI**. Netmiko opens an SSH session, sends a command, waits for the prompt and returns the output as a `str`.

Part	Meaning
`device_type`	CLI flavour: `cisco_xe` for CUBE on IOS-XE
`ConnectHandler(**d)`	log in; `**` unpacks the dict (1.6.1)
`with … as conn`	logs out automatically at the end
`send_command(cmd)`	run one `show` command, get the text

× COMMON MISTAKE

Without `with`, call `conn.disconnect()` yourself. Forgotten sessions stay open until the VTY lines on the router run out.

Turning show output into data



TextFSM templates turn known commands into a list of dicts

```
import re

out = conn.send_command("show call active voice brief")
m = re.search(r"Total call-legs:\s*(\d+)", out)
legs = int(m.group(1)) if m else 0
calls = legs // 2          # one call = two legs on CUBE

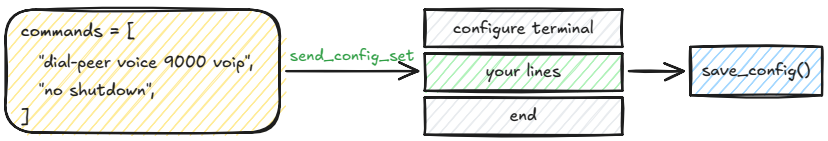
rows = conn.send_command("show ip interface brief",
                         use_textfsm=True)
rows[0]   # {'interface': 'GigabitEthernet1', 'status': 'up', …}
```

CLI output is text made for humans. Picking a value by line number and column (`splitlines()[13].split()[4]`) breaks as soon as a software upgrade adds one line.

- **Regex** (`re`) finds a value by its label: `Total call-legs:` works wherever that line moves.
- **TextFSM** (`use_textfsm=True`) parses known commands into a list of dicts using the ntc-templates collection.

> **! WATCH OUT**
> ntc-templates covers many routing commands but almost no voice commands (only `show isdn status` today). For CUBE, plan on regex.

Changing configuration



enter and leave config mode are added for you

```python
def alternate_announcement(conn, enable):
    commands = [
        "dial-peer voice 9000 voip",
        "no shutdown" if enable else "shutdown",
    ]
    out = conn.send_config_set(commands)
    if "% Invalid" in out:
        raise RuntimeError(out)
    conn.save_config()              # write memory

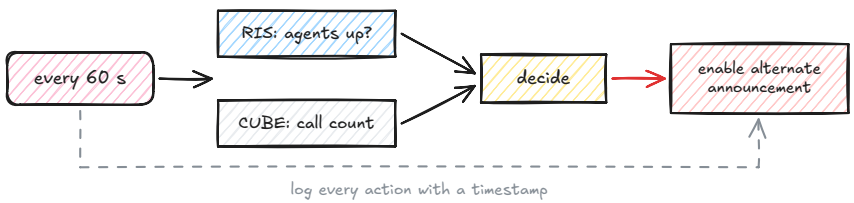
with ConnectHandler(**cube) as conn:
    alternate_announcement(conn, enable=True)
    print(conn.send_command("show dial-peer voice summary"))
```

`send_config_set` takes a **list** of config lines. It enters configuration mode, sends every line and leaves again. `save_config()` then writes the running config to startup.

The router does not raise an error for a wrong command: it answers with `% Invalid input`. Check the returned text, and verify the result with a `show` command.

> **× COMMON MISTAKE**
> Do not put `"conf t"` into the list: Netmiko already entered config mode, so the extra line only adds noise or an error.

Scheduled checks · RIS + CUBE




```
import time
from netmiko import (NetmikoTimeoutException,
                     NetmikoAuthenticationException)

agents = ["CSFAGENT01", "CSFAGENT02"]

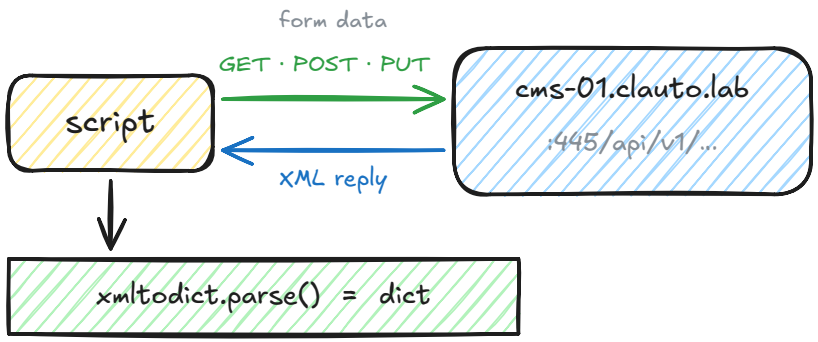
while True:
    status = ris_status(agents)                  # 2.5
    up = any(s == "Registered" for s, _ in status.values())
    try:
        with ConnectHandler(**cube) as conn:
            alternate_announcement(conn, enable=not up)
    except (NetmikoTimeoutException,
            NetmikoAuthenticationException) as e:
        print(time.asctime(), "CUBE not reachable:", e)
    time.sleep(60)
```

Scripts become most useful when they combine sources: RIS knows whether the contact-center agents are registered, the CUBE decides where calls go. A loop with `time.sleep()` (or the `schedule` package) runs the check every minute.

Netmiko raises its own exceptions for a host that does not answer and for a wrong login. Catch them, log them, and keep the loop alive.

> ▸ **IN THE COLLAB WORLD**
> If no agent's Jabber is registered, enable the dial peer that sends callers to an alternate announcement. When an agent is back, shut it again.

Cisco Meeting Server API




```python
import os
import requests
import xmltodict                    # pip install xmltodict

BASE = "https://cms-01.clauto.lab:445/api/v1"
s = requests.Session()
s.auth = (os.environ["CMS_USER"], os.environ["CMS_PASSWORD"])
s.verify = "certs/clauto-ca.pem"

r = s.get(f"{BASE}/system/status", timeout=10)
r.raise_for_status()
status = xmltodict.parse(r.content)["status"]
status["softwareVersion"], status["callLegsActive"]
# ('3.10', '2')
```

CMS has a REST API on its **Web Admin** port (often 445). It uses Basic auth, sends **form data** for writes and answers in **XML**, not JSON.

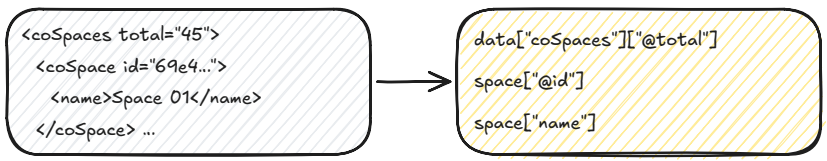
`xmltodict` turns the XML into nested dicts, so the rest feels like working with JSON from Webex.

Object	Endpoint
spaces	`/coSpaces` , `/coSpaces/<id>`
active meetings	`/calls`
participants	`/calls/<id>/callLegs`
server health	`/system/status`

▸ **IN THE COLLAB WORLD**

Create an **API user** in CMS for scripts (`api_svc`), separate from the admin login, and keep it in `.env` (1.11).

Example 1 · list all spaces



attributes get an @ prefix, elements become keys

```python
def list_spaces(name_filter=""):
    spaces, offset = [], 0
    while True:
        params = {"offset": offset, "limit": 20,
                  "filter": name_filter}
        r = s.get(f"{BASE}/coSpaces", params=params, timeout=10)
        r.raise_for_status()
        xml = xmltodict.parse(r.content, force_list=("coSpace",))
        data = xml["coSpaces"]
        page = data.get("coSpace", [])
        spaces += page
        offset += len(page)
        if not page or offset >= int(data["@total"]):
            return spaces

for sp in list_spaces():
    print(sp["@id"], sp["name"])
```

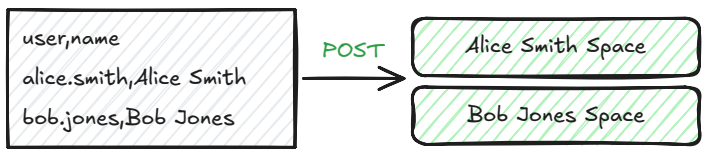
CMS returns lists in small pages. The root element carries the `total` count. Move `offset` forward page by page until you have `total` items.

`filter` searches by name on the server, so you do not download every space to find one.

× COMMON MISTAKE

With one result, `xmltodict` returns a single `dict` instead of a `list`, and the `for` loop walks over its keys. Pass `force_list=("coSpace",)` to always get a list.

Example 2 · create spaces from a CSV




```python
import csv

with open("users.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        form = {"name": f"{row['name']} Space",
                "uri": f"{row['user']}.space",
                "passcode": "4711"}
        r = s.post(f"{BASE}/coSpaces", data=form, timeout=10)
        if r.status_code != 200:
            print("failed:", row["user"], r.text)
            continue
        space_id = r.headers["Location"].split("/")[-1]
        print("created", form["uri"], space_id)

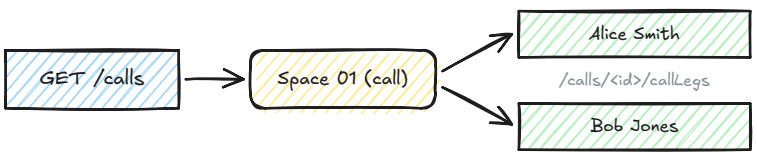
s.put(f"{BASE}/coSpaces/{space_id}", data={"passcode": "2026"})
s.delete(f"{BASE}/coSpaces/{space_id}")
```

Writes send **form data**: `data=` (not `json=`) in `requests`. A successful `POST` returns an empty body; the new object's id is in the `Location` header.

Call	Effect
`POST /coSpaces`	create a space
`PUT /coSpaces/<id>`	change only the fields you send
`DELETE /coSpaces/<id>`	remove the space

Errors come back as `400` with a `<failureDetails>` element, for example `duplicateCoSpaceUri`: log it and go on (1.7).

Example 3 · who is in which meeting




```python
def get_xml(path, key, item):
    r = s.get(f"{BASE}{path}", timeout=10)
    r.raise_for_status()
    return xmltodict.parse(r.content, force_list=(item,))[key]

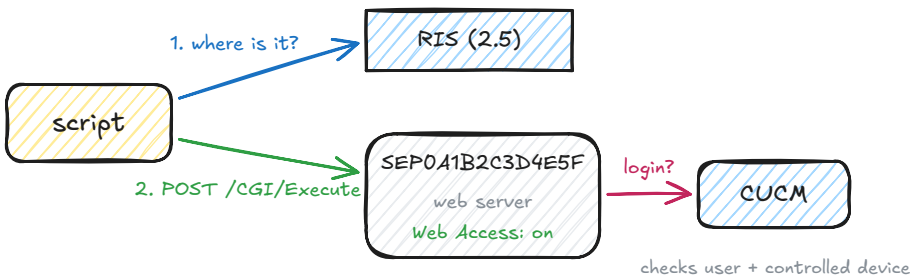
calls = get_xml("/calls", "calls", "call")
for call in calls.get("call", []):
    legs = get_xml(f"/calls/{call['@id']}/callLegs",
                   "callLegs", "callLeg")
    print(f"{call['name']}: {legs['@total']} participants")
    for leg in legs.get("callLeg", []):
        print("   ", leg["name"], leg["remoteParty"])
# Space 01: 2 participants
#     Alice Smith alice.smith@clauto.lab
#     Bob Jones bob.jones@clauto.lab
```

An active meeting is a **call**; every participant in it is a **call leg**. Two nested requests give a live meeting report.

A small helper like `get_xml()` keeps the parsing in one place, so every new report is just a few lines.

> ▸ **IN THE COLLAB WORLD**
> Run it before maintenance: if no call is active, CMS can be restarted without dropping a meeting. Add it to the daily RIS report (2.5) for one morning overview.

The phone's own web server




```
POST http://<phone>/CGI/Execute
Content-Type: application/x-www-form-urlencoded

XML=<CiscoIPPhoneExecute>
      <ExecuteItem URL="Key:Settings"/>
    </CiscoIPPhoneExecute>
```

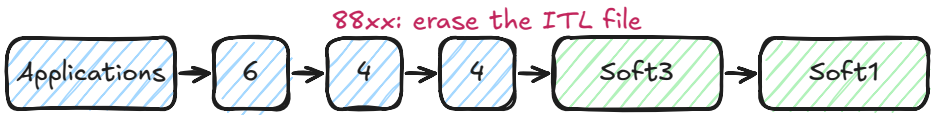
```
def phone_hosts(names):     # ris_status (2.5), but
    ...                     # keeps the address
    for dev in devs:
        ip = dev["IPAddress"]["item"][0]["IP"]
        hosts[dev["Name"]] = ip
    return hosts
```

Some jobs have no API at all: erasing the ITL file, resetting a menu setting, taking a picture of the screen. But every Cisco IP phone runs a small web server, and `/CGI/Execute` lets a script **press keys** or **dial** as if someone stood in front of the phone.

Prerequisite	Where
Web Access enabled	phone config in CUCM
end user with the phone	user's **Controlled Devices**
phone address	from RIS (`IPAddress`), never typed by hand

The phone asks CUCM to check the login, so a normal end user that owns the phone is enough. The reply is a small XML document: `<ResponseItem … Status="0"/>` means success.

Pressing keys remotely



one POST per key press, with a pause so the menu can open

```python
import time
import xml.etree.ElementTree as ET
import requests

def press(host, item, wait=1):
    xml = f'<CiscoIPPhoneExecute><ExecuteItem URL="{item}"/>' \
          '</CiscoIPPhoneExecute>'
    r = requests.post(f"http://{host}/CGI/Execute",
                      data={"XML": xml}, auth=auth, timeout=10)
    r.raise_for_status()
    time.sleep(wait)                        # let the menu open
    return ET.fromstring(r.content).find("ResponseItem") \
             .get("Status") == "0"

ITL_88XX = ["Key:Applications", "Key:KeyPad6", "Key:KeyPad4",
            "Key:KeyPad4", "Key:Soft3", "Key:Soft1"]
for name, host in phone_hosts(phones).items():   # from RIS
    ok = all(press(host, key) for key in ITL_88XX)
    print(name, "done" if ok else "check manually")
```

Each `ExecuteItem` is one action. A key sequence is a list, sent one `POST` at a time with a short pause, exactly like a person walking through the menu.

Item	Action
`Key:Settings` `Key:Applications`	hard keys
`Key:KeyPad0` … `Key:KeyPad9`	dial pad
`Key:Soft1` … `Key:Soft4`	soft keys under the screen
`Dial:1002`	place a call

! WATCH OUT

Menu paths differ by model and firmware: keep one key list per model and test it on one phone first. The script cannot see the screen, so a missed step goes unnoticed (next slide).

Screenshots · CGI/Screenshot

```python
from pathlib import Path

def screenshot(name, host):
    r = requests.get(f"http://{host}/CGI/Screenshot",
                     auth=auth, timeout=10)
    r.raise_for_status()
    out = Path("shots") / f"{name}.png"
    out.parent.mkdir(exist_ok=True)
    out.write_bytes(r.content)      # bytes, 1.3.10
    return out

for name, host in phone_hosts(phones).items():
    try:
        print(screenshot(name, host))
    except requests.exceptions.RequestException as e:
        print(name, "failed:", e)
# shots/SEP0A1B2C3D4E5F.png
```

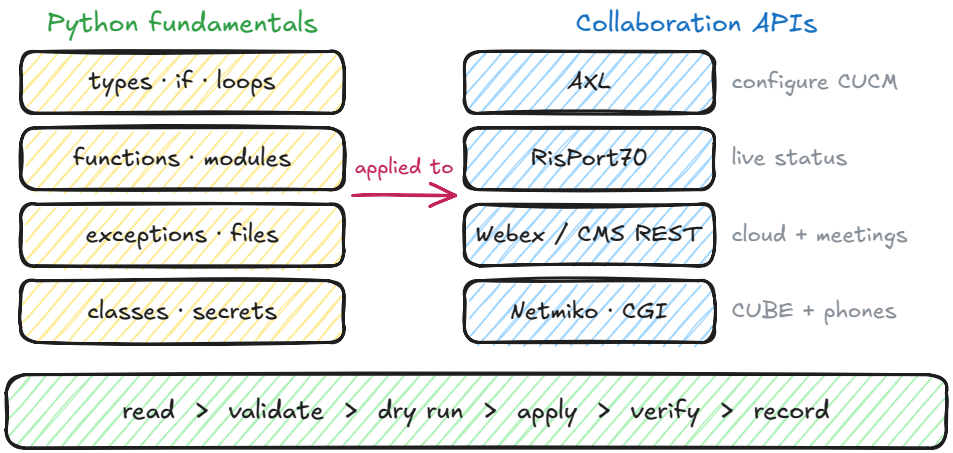
A `GET` on `/CGI/Screenshot` returns the current screen as an image. `r.content` holds the raw `bytes`, written straight to a `.png` file.

Use it to **verify** what the key presses did, or to document a problem without visiting the desk: “is the line registered”, “which error is on the screen”.

> ▸ **IN THE COLLAB WORLD**
> After a bulk ITL erase, take a screenshot of every phone and scroll through the folder: one look shows which phones did not reach the expected menu.

> ★ **TIP**
> Working reference: `erase_itl.py` in the course repository holds tested key sequences for 69xx, 78xx, 88xx and 99xx models.

What you can do now



In five days you went from the first `print` to scripts that read, change and check a whole collaboration cluster:

- **Python** as the language: data, decisions, loops, functions, files and errors.
- **APIs** as the doors: AXL and RIS on CUCM, REST on Webex and CMS, SSH on CUBE, the web server on every phone.
- **A safe way of working**: secrets outside the code, dry run first, verify after every change, keep a record.

> ▸ **IN THE COLLAB WORLD**
> Start small: one read-only report that runs every morning. Every later script reuses the same connection, loop and error handling.

Thank you

Questions, ideas, scripts you want to build next?

Keep practising

- Turn one weekly manual task into a script.
- Add `--apply`, a dry run and a result CSV to it.
- Share it with your team in a git repository.

References

- Course scripts: `github.com/ilkermansur/collab_automation`
- AXL & RisPort70: `developer.cisco.com/docs/axl`
- Webex APIs: `developer.webex.com`
- Python docs: `docs.python.org/3`